

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn chân thành nhất tới TS. Đỗ Bá Lâm. Trong suốt quá trình thực hiện đề tài, thầy đã luôn dành thời gian định hướng khoa học, tận tình hướng dẫn và hỗ trợ em vượt qua những khó khăn từ giai đoạn bắt đầu cho đến khi hoàn thành đồ án tốt nghiệp này.

Đồng thời, em xin trân trọng cảm ơn tập thể giảng viên Đại học Bách khoa Hà Nội nói chung và các thầy cô giáo tại Trường Công nghệ Thông tin và Truyền thông nói riêng. Những kiến thức chuyên môn sâu sắc cùng phương pháp luận nghiên cứu khoa học mà các thầy cô truyền đạt trong suốt quá trình học tập là hành trang vô cùng quý giá cho em.

Cuối cùng, em xin gửi lời cảm ơn đến gia đình và bạn bè. Đây là nguồn động viên tinh thần to lớn, luôn chia sẻ, đồng hành và tạo mọi điều kiện thuận lợi nhất để em yên tâm tập trung học tập và hoàn thành chặng đường nghiên cứu này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh rủi ro an ninh mạng ngày càng phức tạp, việc bảo vệ các hệ thống liên lạc nội bộ đang trở thành ưu tiên hàng đầu, thúc đẩy sự phổ biến của tiêu chuẩn mã hóa đầu cuối (E2EE) như một giải pháp bảo mật tất yếu. Hiện nay, hầu hết các ứng dụng nhắn tin E2EE hàng đầu vận hành dựa trên giao thức Double Ratchet, một tiêu chuẩn bộc lộ nhiều hạn chế khi mở rộng quy mô nhóm. Nhằm giải quyết bài toán này, Messaging Layer Security (MLS) ra đời như một giao thức tiên phong, tái định nghĩa toàn diện kiến trúc mã hóa nhóm. Tuy nhiên, tiêu chuẩn MLS hiện tại được thiết kế độc quyền cho kiến trúc tập trung và bắt buộc phụ thuộc vào một máy chủ điều phối để đảm bảo thứ tự. Việc triển khai MLS trên môi trường mạng ngang hàng (P2P) phi tập trung vẫn là một thách thức chưa được giải quyết.

Từ thực tiễn đó, em quyết định theo đuổi hướng nghiên cứu tích hợp trực tiếp giao thức MLS lên kiến trúc mạng ngang hàng, tước bỏ hoàn toàn máy chủ trung tâm. Hướng đi này được lựa chọn nhằm chứng minh khả năng dung hợp giữa tiêu chuẩn mã hóa nhóm tiên tiến nhất hiện nay và triết lý mạng phi tập trung, từ đó vạch ra mô hình lý thuyết cho một hệ thống truyền thông không tồn tại điểm lỗi duy nhất.

Tổng quan giải pháp của nghiên cứu là đề xuất một lớp điều phối phi tập trung. Kiến trúc này thay thế hoàn toàn vai trò của máy chủ trong việc quản lý cập nhật trạng thái nhóm, đồng thời cung cấp khả năng tự động duy trì trạng thái mã hóa nhóm, nhận diện và hàn gắn phân nhánh khi mạng xảy ra đứt gãy. Bên cạnh phần thiết kế giao thức, đồ án còn xây dựng một ứng dụng desktop nhằm hiện thực các luồng khởi tạo định danh, cấp bundle, tham gia tổ chức, trao đổi thông điệp mã hóa đầu cuối và quản trị thành viên. Sự hiện diện của ứng dụng này cho phép đối chiếu trực tiếp giữa mô hình giao thức và hành vi vận hành ở mức ứng dụng, qua đó củng cố tính khả thi của hướng tiếp cận được đề xuất.

Sinh viên thực hiện

Lê Tiến Dũng

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện tại và hạn chế	2
1.3 Mục tiêu và định hướng giải pháp	3
1.4 Đóng góp của đề án	4
1.5 Bố cục đề án	4
CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT	6
2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán	6
2.2 Các kết quả nghiên cứu tương tự	8
2.2.1 Các cơ chế mã hóa nhóm trước MLS.....	8
2.2.2 MLS với Delivery Service	9
2.2.3 Các hướng phi tập trung hóa MLS	9
2.3 Nền tảng MLS và TreeKEM	10
2.3.1 Các khái niệm cốt lõi: client, group, member và epoch	10
2.3.2 Ratchet Tree và TreeKEM.....	10
2.3.3 Proposal, Commit và tiến trình chuyển epoch	11
2.3.4 Các dấu hiệu nhận biết trạng thái: tree hash, transcript hash và epoch authenticator	11
2.3.5 Application messages, delayed messages và External Commit.....	11
2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán	12
2.4.1 Đặc tính của mạng ngang hàng.....	12
2.4.2 go-libp2p và GossipSub.....	12
2.4.3 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng.....	13
2.4.4 Khoảng trống cần giải quyết.....	13

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT.....	15
3.1 Tổng quan bài toán và hướng tiếp cận	15
3.1.1 Bài toán cần giải quyết	15
3.1.2 Kiến trúc logic của giải pháp.....	16
3.1.3 Các cơ chế chính của phương pháp	16
3.2 Giao thức người ghi duy nhất theo Epoch.....	17
3.2.1 Mục tiêu thiết kế	17
3.2.2 Tập ứng viên hợp lệ	17
3.2.3 Hàm bầu chọn Token Holder	18
3.2.4 Tạo Proposal và Commit.....	19
3.2.5 Chuyển quyền và failover	20
3.3 Kiểm tra nhất quán Epoch	20
3.3.1 Mục tiêu thiết kế	20
3.3.2 Lớp bao điều phối ở mức khái niệm	20
3.3.3 Phân loại thông điệp theo Epoch.....	21
3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai	22
3.3.5 Xử lý thao tác bị bỏ lỡ	22
3.4 Cơ chế phục hồi phân nhánh trạng thái	23
3.4.1 Bối cảnh network partition	23
3.4.2 Phát hiện nghi vấn phân nhánh và đối chiếu trạng thái	23
3.4.3 Hàm trọng số nhánh	24
3.4.4 Quy trình hàn gắn phân nhánh.....	25
3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock	26
3.5.1 Lý do cần HLC	26
3.5.2 Tạo timestamp khi gửi tin.....	26
3.5.3 Cập nhật HLC khi nhận tin	27

3.5.4 Vai trò của HLC trong hệ thống.....	28
3.6 Hiện thực ứng dụng thử nghiệm phục vụ kiểm chứng.....	28
3.6.1 Ranh giới trách nhiệm trong ứng dụng thử nghiệm	28
3.6.2 Vai trò của ứng dụng desktop	29
3.7 Tính đúng đắn trực giác của phương pháp	29
CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT.....	31
4.1 Mô hình phân tích và giả định.....	31
4.2 Bất biến 1: Epoch cục bộ chỉ được giữ nguyên hoặc tăng.....	32
4.2.1 Phát biểu bất biến.....	32
4.2.2 Cơ sở từ MLS	32
4.2.3 Cơ chế bảo toàn bất biến.....	32
4.3 Bất biến 2: Với cùng một view, chỉ có một Token Holder.....	33
4.3.1 Phát biểu bất biến.....	33
4.3.2 Ý nghĩa đối với concurrent commits	34
4.4 Bất biến 3: Phân nhánh phải được nhận biết sau bước đối chiếu lịch sử.....	34
4.4.1 Phân biệt chậm đồng bộ và phân nhánh thực sự.....	34
4.4.2 Cam kết cho tiền tổ lịch sử Commit	34
4.5 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh....	36
4.5.1 Vì sao không thể gộp trực tiếp hai nhánh.....	36
4.5.2 Quy tắc chọn nhánh.....	36
4.5.3 External Join như cơ chế hội tụ.....	37
4.6 Xử lý thông điệp phát sinh ở nhánh thua	37
4.7 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit.....	38
4.7.1 Phân biệt hai loại thứ tự	38
4.7.2 Cơ sở của HLC	38

4.7.3 Giới hạn của HLC	38
4.8 Phân tích chi phí lý thuyết	39
4.8.1 Chi phí gửi application message	39
4.8.2 Chi phí cập nhật thành viên và rekey	39
4.8.3 Chi phí của Single-Writer	39
4.8.4 Chi phí của kiểm tra epoch và đối chiếu trạng thái.....	39
4.8.5 Chi phí của Fork Healing	40
4.9 Tổng hợp các bất biến và cơ chế bảo toàn.....	40
4.10 Giới hạn lý thuyết của phương pháp.....	41
CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM.....	42
5.1 Mục tiêu và phạm vi đánh giá	42
5.2 Môi trường và phương pháp thí nghiệm	42
5.3 Đối chiếu trên ứng dụng desktop	43
5.3.1 Tạo tổ chức và cấp bundle	44
5.3.2 Thiết bị mới vào hệ thống	44
5.3.3 Trao đổi tin nhắn trong nhóm	44
5.4 Đánh giá hội tụ trong điều kiện mạng hỗn loạn.....	46
5.5 Đánh giá khả năng phục hồi sau phân vùng mạng.....	46
5.6 Đánh giá cơ chế một người ghi và gom đề xuất.....	48
5.7 Đánh giá khả năng mở rộng của thao tác thành viên.....	49
5.8 Đánh giá chi phí mật mã MLS	49
5.9 Chi phí phụ trợ của lớp điều phối	51
5.10 Kiểm chứng Forward Secrecy bằng thành phần MLS thật.....	52
5.11 Nhận xét và giới hạn đánh giá	52
CHƯƠNG 6. KẾT LUẬN	54
6.1 Kết luận chung	54

6.2 Giới hạn của đồ án.....	55
6.3 Hướng phát triển.....	55
TÀI LIỆU THAM KHẢO.....	58
PHỤ LỤC.....	60
A. ỨNG DỤNG DEMO VÀ KỊCH BẢN MINH HỌA.....	60
A.1 Những phần đã triển khai trong ứng dụng desktop	60
A.2 Chuỗi tham gia hệ thống và cấp bundle.....	61
A.3 Không gian làm việc và cộng tác	62
A.4 Quản trị nhóm và chẩn đoán	64

DANH MỤC HÌNH VẼ

Hình 2.1	Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS .	7
Hình 3.1	Kiến trúc logic của giải pháp và ranh giới trách nhiệm giữa lớp điều phối với lớp MLS	16
Hình 5.1	Cấp bundle cho thiết bị mới trên ứng dụng desktop	44
Hình 5.2	Trao đổi tin nhắn trong nhóm	45
Hình 5.3	Màn hình quản trị thành viên của nhóm	45
Hình 5.4	Hội tụ epoch của các nút trong chaos test	46
Hình 5.5	Thời gian phục hồi khi độ dài phân vùng thay đổi	47
Hình 5.6	Hiệu quả của cơ chế gom đề xuất khi mức đồng thời tăng . . .	48
Hình 5.7	Phân rã chi phí xử lý cục bộ giữa phần MLS, phần lưu trữ và phần điều phối	51
Hình A.1	Khởi tạo định danh và tạo tổ chức gốc trong ứng dụng thử nghiệm	61
Hình A.2	Chuỗi cấp bundle cho thiết bị mới trong ứng dụng thử nghiệm	61
Hình A.3	Tổng quan không gian làm việc của ứng dụng desktop thử nghiệm	62
Hình A.4	Trao đổi tin nhắn trong cùng một nhóm ở các thiết bị khác nhau	63
Hình A.5	Bảng hoạt động và không gian kênh của ứng dụng thử nghiệm	63
Hình A.6	Các hộp thoại tạo nhóm và thêm thành viên	64
Hình A.7	Màn hình quản trị thành viên và chính sách mời của nhóm . .	64
Hình A.8	Các màn hình theo dõi trạng thái mạng và trạng thái nhóm . .	65

DANH MỤC BẢNG BIỂU

Bảng 4.1	Quy tắc xử lý thông điệp theo epoch	32
Bảng 4.2	Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn	40
Bảng 5.1	Các tiêu chí đánh giá thực nghiệm	43
Bảng 5.2	Thời gian phục hồi sau phân vùng mạng	47
Bảng 5.3	So sánh xử lý ngay và cơ chế gom đề xuất	48
Bảng 5.4	Chi phí thao tác thành viên theo quy mô nhóm	49
Bảng 5.5	So sánh trung vị thời gian xử lý giữa hai hướng mã hóa, với payload 1024 byte và 100 mẫu cho mỗi điểm đo	50
Bảng 5.6	So sánh p95 và p99 ở nhóm 4096 thành viên	50

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
BFT	Chịu lỗi Byzantine (Byzantine Fault Tolerance)
DHT	Bảng băm phân tán (Distributed Hash Table)
DMLS	Bảo mật tầng thông điệp phi tập trung (Decentralized Messaging Layer Security)
DS	Dịch vụ phân phối (Delivery Service)
E2EE	Mã hóa đầu cuối (End-to-End Encryption)
FREEK	Thỏa thuận khóa nhóm liên tục chịu rẽ nhánh (Fork-Resilient Continuous Group Key Agreement)
FS	Bảo mật xuôi (Forward Secrecy)
HLC	Đồng hồ logic hỗn hợp (Hybrid Logical Clock)
IP	Giao thức Internet (Internet Protocol)
LAN	Mạng cục bộ (Local Area Network)
mDNS	Hệ thống phân giải tên miền multicast (multicast Domain Name System)
MLS	Bảo mật tầng thông điệp (Messaging Layer Security)
NAT	Biến đổi địa chỉ mạng (Network Address Translation)
P2P	Mạng ngang hàng (Peer-to-Peer)
PCS	Bảo mật sau thỏa hiệp (Post-Compromise Security)
QUIC	Kết nối Internet UDP nhanh (Quick UDP Internet Connections)
RFC	Tài liệu đặc tả chuẩn / Yêu cầu bình luận (Request for Comments)

Thuật ngữ	Ý nghĩa
TCP	Giao thức điều khiển truyền vận (Transmission Control Protocol)
WAN	Mạng diện rộng (Wide Area Network)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh hoạt động trao đổi thông tin ngày càng phụ thuộc vào hạ tầng số, yêu cầu bảo vệ nội dung liên lạc nội bộ đang trở nên cấp thiết đối với cơ quan, doanh nghiệp và các tổ chức có dữ liệu nhạy cảm. Rò rỉ thông tin không chỉ gây thiệt hại về kinh tế, mà còn ảnh hưởng trực tiếp tới uy tín và an toàn vận hành. Vì vậy, mã hóa đầu cuối đã trở thành một hướng tiếp cận quan trọng, trong đó chỉ các thiết bị đầu cuối hợp lệ mới có khả năng giải mã nội dung thông điệp, còn các thành phần trung gian không được phép truy cập bản rõ [1], [2].

Tuy nhiên, trong nhiều hệ thống nhắn tin bảo mật hiện nay, máy chủ trung tâm vẫn giữ vai trò rất lớn trong quá trình vận hành. Máy chủ thường đảm nhiệm việc định tuyến thông điệp, hỗ trợ thiết bị ngoại tuyến, lưu trữ vật liệu khóa ban đầu và đồng bộ trạng thái giữa các thành viên [2]. Cách tổ chức này giúp hệ thống dễ quản lý hơn, nhưng đồng thời cũng tạo ra sự phụ thuộc đáng kể vào hạ tầng tập trung. Với những môi trường như mạng nội bộ biệt lập, mạng tạm thời không có Internet, hoặc các tổ chức muốn kiểm soát chặt chẽ dữ liệu, sự phụ thuộc đó trở thành một hạn chế thực tế. Những bối cảnh này đòi hỏi một mô hình trong đó dữ liệu được ưu tiên lưu tại thiết bị, các nút có thể trao đổi trực tiếp với nhau, và hệ thống vẫn duy trì được hoạt động ngay cả khi không có máy chủ trung tâm luôn sẵn sàng. Định hướng đó gần với tinh thần ưu tiên dữ liệu cục bộ và mạng ngang hàng [3].

Đối với liên lạc hai bên, nhiều giao thức đã chứng minh được hiệu quả trong thực tế, tiêu biểu là Double Ratchet. Tuy nhiên, khi mở rộng sang liên lạc nhóm, bài toán không còn dừng ở việc mã hóa từng thông điệp, mà chuyển thành bài toán duy trì một trạng thái mật mã chung giữa nhiều thiết bị có thể gửi, nhận, mất kết nối và quay trở lại ở những thời điểm khác nhau [1], [4]. Nói cách khác, khó khăn cốt lõi của truyền thông nhóm bảo mật không chỉ nằm ở bí mật nội dung, mà còn nằm ở việc bảo đảm tất cả thành viên hợp lệ cùng nhìn thấy một trạng thái khóa nhóm nhất quán.

Messaging Layer Security (MLS) được chuẩn hóa trong RFC 9420 để giải quyết bài toán thiết lập khóa nhóm bất đồng bộ với các bảo đảm quan trọng như Forward Secrecy và Post-Compromise Security [1]. Nhờ cấu trúc TreeKEM, MLS cho phép cập nhật trạng thái khóa nhóm hiệu quả hơn nhiều so với các cơ chế mã hóa nhóm dựa hoàn toàn trên các kênh hai bên. Dù vậy, MLS không phải là một hệ thống nhắn tin hoàn chỉnh. Theo kiến trúc của RFC 9750, giao thức này vẫn cần được đặt trong một môi trường triển khai cụ thể, nơi ứng dụng phải tự giải quyết các vấn đề

như xác thực danh tính, phân phối thông điệp và đặc biệt là xử lý thứ tự các bản tin làm thay đổi trạng thái nhóm [2].

Chính ở điểm này xuất hiện mâu thuẫn trung tâm của đề tài. Trong mô hình có Delivery Service nhất quán mạnh, thành phần phân phối có thể đóng vai trò áp đặt thứ tự tuyến tính cho các Commit và loại bỏ xung đột khi nhiều thành viên cùng cập nhật nhóm tại một epoch. Ngược lại, trong môi trường P2P không có một thực thể trung tâm làm bộ tuần tự hóa, các Commit có thể đến các thiết bị theo những thứ tự khác nhau. Hệ quả là các nút có thể áp dụng những chuỗi thay đổi không giống nhau và làm nhóm rơi vào trạng thái phân nhánh mật mã, tức là các thành viên tuy vẫn mang cùng định danh nhóm nhưng không còn chia sẻ cùng trạng thái khóa để tiếp tục liên lạc [2], [5], [6].

Từ thực tế đó, đề án lựa chọn nghiên cứu bài toán vận hành MLS trên mạng ngang hàng theo hướng bổ sung một lớp điều phối phi tập trung ở tầng ứng dụng. Mục tiêu của hướng tiếp cận này không phải là sửa đổi lõi mật mã của MLS, mà là xây dựng các cơ chế hỗ trợ để nhóm vẫn có thể duy trì tính nhất quán trạng thái, phát hiện phân nhánh khi cần thiết và hội tụ trở lại sau khi mạng được khôi phục.

1.2 Các giải pháp hiện tại và hạn chế

Các hướng tiếp cận hiện nay đối với truyền thông nhóm bảo mật có thể chia thành hai nhóm lớn. Nhóm thứ nhất mở rộng từ các cơ chế bảo mật hai bên sang môi trường nhóm; nhóm thứ hai sử dụng giao thức thiết lập khóa nhóm chuyên biệt, trong đó MLS là đại diện tiêu biểu [1], [7], [8]. Mỗi hướng đều có giá trị thực tiễn nhất định, nhưng khi đặt trong yêu cầu vận hành P2P hoàn toàn phi tập trung, các giới hạn của chúng bộc lộ rõ.

Với nhóm thứ nhất, những cơ chế như Sender Keys hay Megolm có ưu điểm là triển khai tương đối thực dụng và có chi phí gửi tin thấp khi nhóm đông thành viên [7], [8]. Tuy nhiên, chúng chủ yếu tối ưu cho việc phát tán thông điệp trong nhóm, chứ không trực tiếp giải quyết bài toán duy trì một trạng thái khóa nhóm nhất quán khi thành viên thay đổi thường xuyên hoặc khi mạng bị chia cắt. Trong các trường hợp khóa gửi tin bị lộ hoặc membership biến động liên tục, chi phí rekey và khả năng phục hồi bảo mật của các cơ chế này vẫn còn những hạn chế đáng kể [7], [8].

Với nhóm thứ hai, MLS cung cấp một mô hình chặt chẽ hơn cho quản lý khóa nhóm và đạt được các bảo đảm bảo mật mạnh hơn cho truyền thông quy mô lớn [1]. Dù vậy, MLS chuẩn vẫn giả định hệ thống triển khai có khả năng bảo đảm việc phân phối và tuần tự hóa các bản tin thay đổi trạng thái [2]. Khi đưa MLS sang môi trường P2P, bài toán khó nhất không nằm ở bản thân thao tác mật mã, mà nằm ở việc thay thế vai trò điều phối vốn thường do Delivery Service đảm nhiệm.

Một số nghiên cứu gần đây, chẳng hạn hướng Decentralized MLS, đã thử giải quyết vấn đề bằng cách mở rộng trực tiếp giao thức MLS để hỗ trợ xử lý Commit ngoài thứ tự trong môi trường phi tập trung [6]. Đây là một hướng nghiên cứu đáng chú ý, nhưng cũng đòi hỏi client duy trì thêm trạng thái mật mã cũ hoặc trạng thái tạm thời, từ đó làm tăng độ phức tạp triển khai và đặt ra thêm đánh đổi đối với Forward Secrecy [5], [6]. Hơn nữa, hướng này hiện vẫn dừng ở mức Internet-Draft, chưa trở thành một chuẩn hoàn chỉnh.

Từ các phân tích trên có thể thấy rằng khoảng trống của bài toán không nằm ở việc thiếu một giao thức mã hóa nhóm, mà nằm ở việc thiếu một cơ chế điều phối đủ nhẹ, đủ thực dụng và phù hợp với đặc tính của mạng ngang hàng. Đây cũng chính là khoảng trống mà đề án hướng tới giải quyết.

1.3 Mục tiêu và định hướng giải pháp

Mục tiêu của đề án là nghiên cứu, thiết kế và hiện thực một cơ chế điều phối phi tập trung nhằm hỗ trợ MLS vận hành trên mạng ngang hàng. Về mặt nghiên cứu, đề án tập trung vào ba vấn đề chính: giảm khả năng xuất hiện concurrent commits khi các nút còn cùng góc nhìn về nhóm, bảo vệ tính nhất quán epoch của trạng thái cục bộ trong môi trường bất đồng bộ, và xây dựng cơ chế phục hồi khi phân nhánh trạng thái vẫn xảy ra do network partition. Về mặt ứng dụng, đề án hướng tới một hệ thống liên lạc nhóm bảo mật ưu tiên dữ liệu cục bộ, có khả năng vận hành trong các điều kiện mạng không ổn định và giảm phụ thuộc vào hạ tầng tập trung [3]. Ứng dụng desktop được xây dựng trong đề án không chỉ đóng vai trò minh họa giao diện, mà còn là môi trường tích hợp để kiểm chứng trọn vẹn chuỗi thao tác từ khởi tạo định danh, cấp bundle, tham gia tổ chức, tạo nhóm cho tới trao đổi thông điệp.

Trên cơ sở mục tiêu đó, đề án lựa chọn cách tiếp cận giữ nguyên lõi mật mã của MLS/OpenMLS, đồng thời bổ sung một lớp điều phối ở phía ngoài để xử lý các vấn đề mà môi trường P2P đặt ra. Thay vì mở rộng trực tiếp giao thức MLS hoặc đưa vào một cơ chế đồng thuận nặng cho mọi Commit, giải pháp đề xuất tập trung vào bốn nội dung: giới hạn quyền tạo Commit theo từng epoch, kiểm tra tính phù hợp của thông điệp trước khi đưa xuống lớp mật mã, phát hiện và hàn gắn phân nhánh trạng thái khi mạng bị chia cắt, và sử dụng đồng hồ logic lai để ổn định thứ tự hiển thị thông điệp ứng dụng. Cách tiếp cận này cho phép hệ thống tận dụng các bảo đảm bảo mật của MLS, đồng thời thích nghi tốt hơn với đặc tính bất đồng bộ và phân tán của mạng ngang hàng.

Trong lựa chọn thiết kế này, đề án chấp nhận một đánh đổi có chủ đích. Hệ thống không theo đuổi nhất quán mạnh tuyệt đối tại mọi thời điểm, bởi điều đó thường

đòi hỏi chi phí đồng thuận cao và làm giảm tính sẵn sàng của các phân vùng mạng không đủ điều kiện liên lạc [9]. Thay vào đó, đề án ưu tiên khả năng hoạt động cục bộ trong điều kiện mạng chia cắt, nhưng vẫn đặt ra cơ chế để các nút có thể hội tụ trở lại một trạng thái MLS hợp lệ sau khi kết nối được khôi phục. Đây là hướng đi phù hợp hơn với mục tiêu P2P và định hướng ưu tiên dữ liệu cục bộ của hệ thống.

1.4 Đóng góp của đề án

Trên cơ sở định hướng nêu trên, đề án có bốn đóng góp chính. Thứ nhất, đề án xác định bài toán trung tâm của MLS trên mạng ngang hàng dưới góc nhìn nhất quán trạng thái mật mã, qua đó chỉ ra rằng khó khăn cốt lõi không nằm ở việc truyền thông điệp, mà nằm ở việc duy trì một chuỗi trạng thái nhóm hợp lệ khi không còn Delivery Service trung tâm. Thứ hai, đề án đề xuất một lớp điều phối phi tập trung bao quanh MLS, trong đó các cơ chế Single-Writer, kiểm tra epoch, phát hiện phân nhánh và phục hồi sau phân mảnh được phối hợp để giảm nguy cơ fork và hỗ trợ hệ thống hội tụ trở lại. Thứ ba, đề án xây dựng một ứng dụng desktop kết hợp mạng P2P, lưu trữ cục bộ và lõi mật mã MLS/OpenMLS, trong đó các luồng khởi tạo, cấp bundle, quản trị tổ chức và trò chuyện nhóm được dùng để kiểm tra khả năng triển khai của hướng tiếp cận đã đề xuất. Thứ tư, đề án cung cấp các kết quả đánh giá ban đầu về tính hội tụ, khả năng phục hồi sau network partition và tác động của lớp điều phối đối với quá trình vận hành MLS trong môi trường mạng ngang hàng.

1.5 Bố cục đề án

Phần còn lại của báo cáo được tổ chức như sau. Chương 2 trình bày cơ sở lý thuyết của đề tài, bao gồm nền tảng MLS, TreeKEM, vai trò của Delivery Service, các hướng tiếp cận liên quan và những đặc tính của môi trường mạng ngang hàng có ảnh hưởng trực tiếp tới bài toán nhất quán trạng thái. Trên nền tảng đó, Chương 3 trình bày chi tiết giao thức điều phối phi tập trung được đề xuất, làm rõ các thành phần, nguyên tắc hoạt động, cách phối hợp giữa lớp điều phối với lõi mật mã MLS, đồng thời giới thiệu ứng dụng desktop minh họa như môi trường hiện thực của các quyết định thiết kế.

Tiếp theo, Chương 4 phân tích cơ sở lý thuyết của giải pháp theo hướng các bất biến quan trọng, điều kiện hội tụ và giới hạn của phương pháp trong môi trường phân tán bất đồng bộ. Chương 5 trình bày quá trình đánh giá thực nghiệm trên ứng dụng thử nghiệm của đề án, kết hợp giữa dữ liệu đo đạc tự động và các kịch bản thao tác trực tiếp trên ứng dụng để làm rõ khả năng hội tụ, phục hồi sau phân mảnh mạng và chi phí vận hành của hệ thống. Cuối cùng, Chương 6 tổng kết các kết quả đạt được, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo

của đồ án; phần phụ lục tập hợp bộ ảnh giao diện và các kịch bản minh họa chi tiết của ứng dụng này.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

Chương 1 đã trình bày bối cảnh, mục tiêu và định hướng chung của đề tài. Tiếp nối phần mở đầu đó, Chương 2 tập trung làm rõ nền tảng lý thuyết cần thiết cho bài toán vận hành MLS trên mạng ngang hàng. Trọng tâm của chương không phải là liệt kê thật nhiều kiến thức rời rạc, mà là chỉ ra mối liên hệ giữa ba lớp vấn đề: lời mật mã của MLS, đặc tính bất đồng bộ của mạng P2P và yêu cầu duy trì một trạng thái nhóm nhất quán.

Nội dung chương được tổ chức thành bốn phần. Trước hết, chương phân tích ngữ cảnh bài toán để làm rõ vì sao khi đưa MLS từ mô hình có máy chủ trung tâm sang mô hình ngang hàng, khó khăn lớn nhất không nằm ở việc truyền tin mà nằm ở việc giữ cho các thành viên cùng đi trên một chuỗi trạng thái mật mã hợp lệ. Tiếp theo, chương khảo sát các hướng nghiên cứu liên quan nhằm chỉ ra ưu điểm, giới hạn và khoảng trống còn lại. Sau đó, chương trình bày các kiến thức nền tảng quan trọng nhất của MLS và TreeKEM. Cuối cùng, chương giới thiệu ngắn gọn những đặc tính của mạng ngang hàng, cơ chế phát tán thông điệp và công cụ hỗ trợ sắp xếp thông điệp ứng dụng trong hệ phân tán.

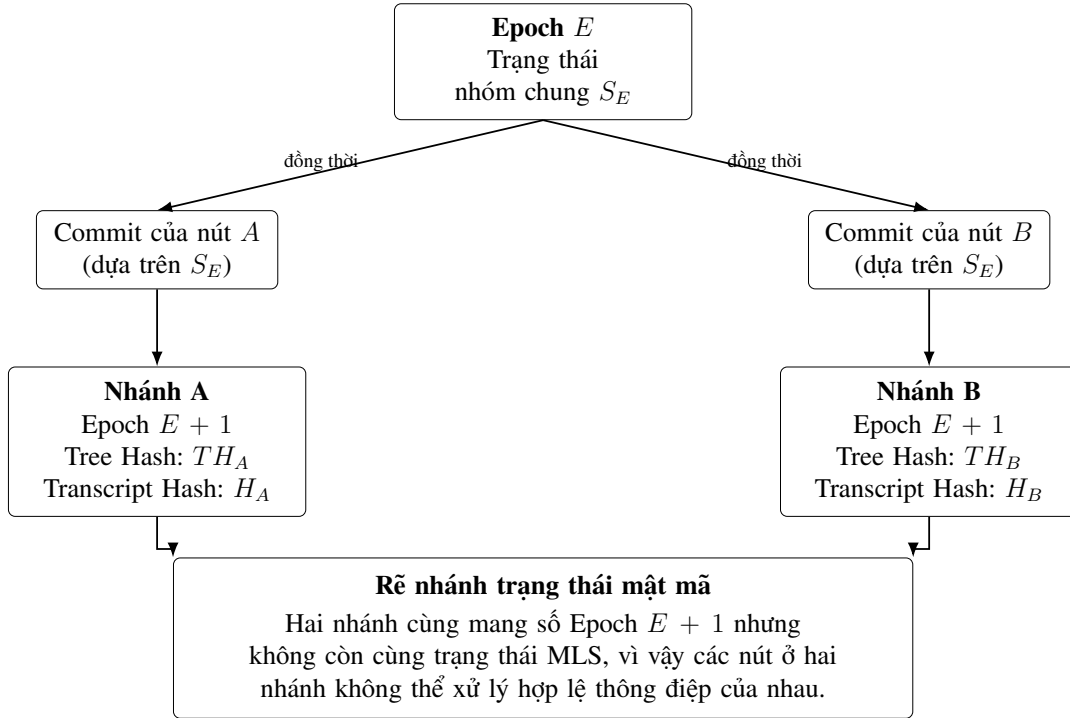
2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán

Trong hệ phân tán nói chung, bài toán nhất quán trạng thái nhằm bảo đảm nhiều nút cùng hiểu và cùng cập nhật dữ liệu theo một quy tắc thống nhất. Đối với hệ thống nhắn tin nhóm được mã hóa đầu cuối, trạng thái cần được giữ nhất quán không chỉ là dữ liệu ứng dụng, mà còn là trạng thái mật mã dùng để sinh khóa, xác thực thành viên và ràng buộc lịch sử hội thoại [1], [2]. Vì vậy, nếu các nút không còn cùng nhìn thấy một trạng thái mật mã chung, khả năng giao tiếp của nhóm sẽ bị ảnh hưởng trực tiếp.

Trong MLS, trạng thái nhóm tiến hóa theo các kỷ nguyên, gọi là epoch. Có thể hiểu mỗi epoch là một phiên bản cụ thể của nhóm, trong đó tập thành viên hợp lệ cùng chia sẻ một bộ bí mật tương ứng. Khi nhóm thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa, trạng thái này phải chuyển sang epoch mới. Nếu tất cả thành viên cùng áp dụng các thay đổi theo cùng một thứ tự, nhóm tiếp tục hoạt động bình thường. Ngược lại, nếu các thay đổi được áp dụng theo những thứ tự khác nhau, các thành viên có thể dần tách khỏi nhau về mặt mật mã [1], [5].

Hiện tượng đó được gọi là rẽ nhánh trạng thái mật mã (Cryptographic State Fork). Đây là tình huống trong đó các nút vẫn nghĩ rằng mình thuộc cùng một nhóm, thậm chí có thể cùng nhìn thấy cùng số epoch, nhưng thực tế lại đang nắm

giữ các bí mật nhóm, hàm băm cây (tree hash) hoặc lịch sử bắt tay (transcript hash) khác nhau. Khi đó, thông điệp được tạo trên một nhánh không còn được xử lý hợp lệ trên nhánh còn lại. Khác với xung đột dữ liệu thông thường, trạng thái mật mã đã rẽ nhánh không thể được ghép lại bằng cách trộn hai phiên bản dữ liệu, vì mỗi nhánh đã sinh ra một chuỗi khóa và một lịch sử xác thực riêng [1], [5], [6]. Hiện tượng này được minh họa trong Hình 2.1.



Hình 2.1: Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS

Trong các triển khai thông thường của MLS, việc hạn chế rẽ nhánh được hỗ trợ bởi Delivery Service (DS), có thể hiểu là lớp dịch vụ tiếp nhận và phân phối các thông điệp MLS giữa các thiết bị [2]. Trong môi trường có DS nhất quán mạnh, khi nhiều thành viên cùng tạo Commit ở cùng một epoch, hệ thống có một điểm tự nhiên để quyết định Commit nào được xử lý trước. Nhờ đó, các thay đổi trạng thái dễ được quan sát theo một thứ tự thống nhất hơn.

Khó khăn xuất hiện khi đưa MLS sang mạng ngang hàng. Trong môi trường P2P, thông điệp có thể đến chậm, đến sai thứ tự, bị lặp lại hoặc tạm thời không đến được do thiết bị ngoại tuyến hay do mạng bị chia cắt. Khi đó, không còn một thực thể trung tâm nào tự nhiên đứng ra sắp xếp thứ tự cho các Commit. Nói cách khác, MLS đòi hỏi chuỗi epoch tiến hóa gần như tuyến tính, trong khi mạng ngang hàng bất đồng bộ lại không tự cung cấp thứ tự tuyến tính toàn cục cho các thông điệp thay đổi trạng thái [2], [9]. Đây chính là mâu thuẫn cốt lõi của bài toán mà đề án cần giải quyết.

Trong phạm vi đồ án, hệ thống được xem như một mạng gồm nhiều thiết bị ngang hàng, mỗi thiết bị lưu cục bộ trạng thái MLS của các nhóm mà nó tham gia. Các tình huống được quan tâm gồm độ trễ mạng không đồng đều, thiết bị ngoại tuyến, sự biến động nút mạng và phân mảnh mạng. Mục tiêu của phần nghiên cứu không phải thay đổi các bảo đảm mật mã nền tảng của MLS, mà là tìm cách tổ chức việc điều phối ở tầng ứng dụng để các thiết bị vẫn có thể hội tụ trở lại một trạng thái nhóm hợp lệ sau những bất ổn của mạng.

2.2 Các kết quả nghiên cứu tương tự

Các nghiên cứu liên quan có thể được nhìn từ ba hướng chính: các cơ chế nhắn tin nhóm xuất hiện trước MLS, MLS trong mô hình có Delivery Service và các nỗ lực đưa MLS sang môi trường phi tập trung. Việc phân chia theo hướng này giúp làm rõ vì sao đồ án không chọn lặp lại hoàn toàn một giải pháp có sẵn, mà cần một hướng tiếp cận riêng phù hợp hơn với mục tiêu P2P và định hướng ưu tiên dữ liệu cục bộ.

2.2.1 Các cơ chế mã hóa nhóm trước MLS

Trước khi MLS được chuẩn hóa, nhiều hệ thống nhắn tin nhóm đã mở rộng từ cơ chế bảo mật hai bên sang môi trường nhiều người tham gia. Một nền tảng quan trọng là Double Ratchet, vốn rất hiệu quả cho liên lạc hai bên [4]. Khi đưa sang nhóm, một hướng phổ biến là để mỗi người gửi tự tạo một khóa gửi tin riêng, rồi phân phối khóa đó cho các thành viên còn lại qua các kênh bảo mật hai bên. Cách làm này thường được gọi là Sender Keys [7]. Sau khi khóa gửi tin đã được chia sẻ, người gửi chỉ cần mã hóa một lần cho mỗi thông điệp, còn hạ tầng bên dưới chịu trách nhiệm phát tán bản mã tới cả nhóm. Ưu điểm của hướng này là chi phí gửi tin thấp và triển khai tương đối thực dụng. Tuy nhiên, khi nhóm đông hoặc thay đổi thành viên thường xuyên, chi phí cập nhật khóa và phục hồi sau thỏa hiệp vẫn tăng đáng kể.

Megolm trong hệ sinh thái Matrix cũng theo tinh thần tối ưu việc gửi tin nhóm [8]. Có thể hiểu đơn giản rằng mỗi người gửi duy trì một phiên gửi ra (*outbound session*), tức một trạng thái khóa được dùng liên tiếp để mã hóa nhiều tin nhắn trong cùng một phòng trò chuyện. Bên trong phiên đó, trạng thái khóa được cập nhật dần theo một cơ chế nhất định của nhóm (*group ratchet*). Thiết kế này giúp việc gửi tin trong nhóm nhẹ hơn, nhưng điểm đổi lại là khả năng phục hồi sau thỏa hiệp không mạnh bằng những mô hình thay khóa nhóm chặt chẽ hơn. Nếu trạng thái của một phiên bị lộ, phạm vi thông điệp bị ảnh hưởng có thể kéo dài cho đến khi hệ thống tạo phiên mới và phân phối lại khóa.

Nhìn chung, Sender Keys và Megolm đều có giá trị thực tiễn cao đối với bài

toán phát tán tin nhắn trong nhóm. Tuy nhiên, chúng không trực tiếp giải quyết bài toán mà đồ án quan tâm nhất, đó là duy trì một trạng thái nhóm nhất quán khi nhiều thiết bị có thể cùng thay đổi trạng thái trong môi trường P2P bất đồng bộ. Đây là lý do MLS trở thành nền tảng phù hợp hơn cho đề tài.

2.2.2 MLS với Delivery Service

MLS được chuẩn hóa để cung cấp cơ chế thiết lập khóa nhóm bất đồng bộ với các bảo đảm bảo mật mạnh hơn cho truyền thông nhóm quy mô lớn [1]. Điểm quan trọng của MLS là nó không chỉ tối ưu việc gửi thông điệp, mà còn mô hình hóa rõ quá trình thay đổi thành viên và cập nhật khóa của cả nhóm thông qua các khái niệm như proposal, commit và epoch.

Trong kiến trúc chuẩn của MLS, Delivery Service thường đảm nhiệm việc lưu trữ một số vật liệu khóa công khai ban đầu, phân phối thông điệp MLS và hỗ trợ quá trình truyền thông giữa các client [2]. Nhờ có một đầu mối như vậy, hệ thống triển khai theo mô hình Client-Server có điều kiện thuận lợi hơn để quan sát các Commit theo một thứ tự đủ nhất quán. Vì thế, MLS hoạt động tự nhiên hơn trong môi trường có DS, nhưng khi bỏ đi điểm điều phối này thì bài toán ordering của Commit lập tức trở nên khó hơn nhiều.

2.2.3 Các hướng phi tập trung hóa MLS

Một hướng tiếp cận trực tiếp là bổ sung một lớp đồng thuận phân tán để thống nhất thứ tự Commit. Các cơ chế như BFT, blockchain hoặc đồng thuận dựa trên quorum có thể tạo ra một nhật ký tuyến tính chung cho các thay đổi trạng thái nhóm. Nếu mọi Commit đều được ghi vào cùng một chuỗi thứ tự, nguy cơ rẽ nhánh trạng thái sẽ giảm rõ rệt. Tuy nhiên, đổi lại là chi phí truyền thông cao, phụ thuộc vào quorum và độ phức tạp triển khai lớn. Với mục tiêu xây dựng một hệ thống P2P nhẹ, có thể vận hành trong mạng nội bộ hoặc mạng không ổn định, đây không phải lựa chọn phù hợp [3], [9].

Một hướng khác là Decentralized MLS (DMLS) của Kohbrok, trong đó chính bản thân MLS được mở rộng để xử lý Commit đến ngoài thứ tự [6]. Về ý tưởng, client giữ lại thêm một phần vật liệu khóa hoặc trạng thái tạm thời để có thể xử lý những nhánh Commit khác nhau khi mạng phi tập trung không cung cấp ordering rõ ràng. Cách làm này cho thấy một hướng nghiên cứu đáng chú ý, nhưng đồng thời cũng làm tăng độ phức tạp triển khai và đặt ra thêm đánh đổi đối với Forward Secrecy [5], [6]. Hơn nữa, hướng này hiện vẫn ở mức Internet-Draft, chưa trở thành chuẩn hoàn chỉnh.

Từ các hướng trên có thể thấy khoảng trống còn lại nằm ở chỗ sau: cần một cơ chế đủ nhẹ để phù hợp với P2P, nhưng vẫn đủ chặt để giảm nguy cơ rẽ nhánh và hỗ

trợ hệ thống hội tụ trở lại sau khi mạng bị chia cắt. Đó cũng là động lực trực tiếp cho hướng tiếp cận mà đề án theo đuổi ở các chương sau.

2.3 Nền tảng MLS và TreeKEM

2.3.1 Các khái niệm cốt lõi: client, group, member và epoch

Trong MLS, client là thực thể nắm giữ khóa mật mã và tham gia vào tiến trình thiết lập trạng thái nhóm. Group là một nhóm logic gồm các client cùng chia sẻ một trạng thái mật mã tại một thời điểm. Member là client đang thực sự nằm trong trạng thái chia sẻ đó và có quyền truy cập các bí mật của nhóm. Epoch, như đã trình bày ở trên, là một phiên bản cụ thể của trạng thái nhóm [1].

Điểm quan trọng cần nhấn mạnh là trạng thái nhóm trong MLS không phải một tập dữ liệu rời rạc có thể ghép lại tùy ý. Nó là một chuỗi epoch nối tiếp nhau, trong đó mỗi epoch mới được sinh ra từ epoch trước đó. Vì vậy, nếu các thành viên không cùng nhìn nhận một chuỗi epoch tương thích, nhóm sẽ mất khả năng duy trì một trạng thái mật mã chung.

2.3.2 Ratchet Tree và TreeKEM

TreeKEM là cơ chế trung tâm giúp MLS cập nhật khóa nhóm hiệu quả. Thay vì để mỗi thành viên phải trao đổi khóa riêng với tất cả các thành viên còn lại, MLS tổ chức các thành viên thành các lá của một cây nhị phân, gọi là Ratchet Tree. Có thể hiểu trực quan rằng mỗi lần nhóm thay đổi, hệ thống chỉ cần cập nhật một số bí mật dọc theo một đường trong cây, thay vì phát lại toàn bộ khóa cho cả nhóm [1].

Khi một thành viên cập nhật khóa, thành viên đó tạo ra một đường dẫn cập nhật (*update path*) từ lá của mình lên gốc cây. Các bí mật mới trên đường đi này được mã hóa cho các nhánh đồng cấp tương ứng, thường được gọi là *copath*. Nhờ cấu trúc cây, số lượng phép tính và lượng dữ liệu cập nhật chỉ tăng gần theo logarit của số thành viên trong nhóm. Nói ngắn gọn, TreeKEM là cơ chế giúp MLS thay khóa nhóm theo cách có cấu trúc và tiết kiệm hơn so với việc phân phối lại khóa theo kiểu tuyến tính cho toàn bộ thành viên.

Cần lưu ý rằng bí mật ở gốc cây không được dùng trực tiếp để mã hóa thông điệp ứng dụng. Sau mỗi Commit, MLS còn đưa các bí mật mới qua một cơ chế sinh khóa (*key schedule*) để tạo ra các khóa cụ thể cho từng mục đích như xác thực, mã hóa thông điệp hay xuất vật liệu khóa [1]. Vì vậy, TreeKEM nên được hiểu là cơ chế cập nhật trạng thái khóa nhóm, còn việc sinh các khóa sử dụng trực tiếp cho thông điệp được xử lý ở bước sau.

2.3.3 Proposal, Commit và tiến trình chuyển epoch

MLS phân biệt rõ proposal và commit. Proposal là thông điệp đề xuất một thay đổi cho nhóm, chẳng hạn thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa. Bản thân proposal chưa làm thay đổi epoch. Commit mới là thông điệp thực sự áp dụng một tập proposal và đưa nhóm sang epoch mới [1].

Nhìn dưới góc độ hệ phân tán, có thể xem Commit như thao tác ghi vào trạng thái mật mã của nhóm. Mỗi Commit đều làm xuất hiện một phiên bản trạng thái mới. Vì thế, nếu tại cùng một epoch mà có nhiều Commit cạnh tranh, nguy cơ rẽ nhánh trạng thái sẽ xuất hiện. Đây là lý do bài toán ordering của Commit luôn giữ vai trò trung tâm khi vận hành MLS trên mạng ngang hàng.

2.3.4 Các dấu hiệu nhận biết trạng thái: tree hash, transcript hash và epoch authenticator

Để kiểm tra các thành viên có còn cùng một trạng thái hay không, MLS sử dụng một số giá trị ràng buộc trạng thái [1]. Trong đó, hàm băm cây (tree hash) phản ánh cấu trúc Ratchet Tree hiện tại. Hàm băm lịch sử bắt tay (transcript hash) tóm tắt lịch sử các bản tin điều khiển của MLS đã được nhóm chấp nhận, chẳng hạn như Proposal và Commit, nhờ đó cho biết các thành viên có thực sự đi qua cùng một chuỗi thay đổi trạng thái hay không. Còn bộ xác thực epoch (epoch authenticator) có thể được dùng để đối chiếu việc các thành viên có thực sự đang chia sẻ cùng trạng thái epoch hay không.

Ý nghĩa của các giá trị này đối với đồ án là rất trực tiếp. Trong môi trường P2P, hai thiết bị có thể cùng báo cùng mã nhóm và cùng số epoch, nhưng điều đó chưa đủ để kết luận rằng chúng còn ở cùng trạng thái MLS. Nếu tree hash hoặc transcript hash khác nhau, về bản chất chúng đã tách thành hai nhánh. Vì vậy, khi nghiên cứu bài toán phát hiện phân nhánh và đồng bộ lại trạng thái, các dấu hiệu nhận biết này đóng vai trò đặc biệt quan trọng.

2.3.5 Application messages, delayed messages và External Commit

MLS phân biệt thông điệp bắt tay (*handshake messages*) với thông điệp ứng dụng (*application messages*) [1]. Proposal và Commit thuộc nhóm thứ nhất vì chúng làm thay đổi hoặc chuẩn bị thay đổi trạng thái nhóm. Ngược lại, application messages là các tin nhắn ứng dụng được mã hóa bằng khóa sinh ra từ trạng thái của epoch hiện tại.

Trong mạng thực tế, thông điệp ứng dụng có thể đến muộn hoặc đến sai thứ tự. MLS có cơ chế hỗ trợ một mức độ nhất định cho các thông điệp bị trễ, chẳng hạn thông qua bộ khóa của người gửi (*sender ratchet*) và bộ đếm thế hệ (*generation*

counter). Tuy nhiên, để bảo vệ Forward Secrecy, thiết bị không nên giữ các khóa cũ vô thời hạn. Điều này tạo ra một đánh đổi quen thuộc: nếu xóa khóa cũ quá sớm thì thông điệp đến muộn có thể không giải mã được; nếu giữ khóa cũ quá lâu thì phạm vi thiệt hại khi thiết bị bị thỏa hiệp sẽ tăng [1].

MLS cũng hỗ trợ cơ chế gia nhập từ bên ngoài, thường được gọi là External Commit hoặc External Join [1]. Có thể hiểu đây là cách để một client chưa ở trong trạng thái hiện tại của nhóm gia nhập lại nhóm khi có đủ thông tin công khai cần thiết. Cơ chế này quan trọng đối với đồ án vì khi trạng thái đã rẽ nhánh, một thiết bị có thể cần từ bỏ nhánh cũ và tham gia lại nhánh đang được chọn.

2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán

2.4.1 Đặc tính của mạng ngang hàng

Mạng ngang hàng khác với mô hình Client-Server ở chỗ không có một máy chủ trung tâm duy nhất điều phối toàn bộ luồng truyền thông. Các thiết bị có thể tự phát hiện nhau, thiết lập kết nối trực tiếp hoặc gián tiếp và trao đổi dữ liệu qua nhiều đường truyền khác nhau. Cách tổ chức này giúp hệ thống giảm phụ thuộc vào hạ tầng trung tâm, nhưng đồng thời làm cho việc điều phối trở nên khó hơn.

Trong môi trường P2P, các hiện tượng như thiết bị ngoại tuyến, độ trễ không đồng đều, trùng lặp thông điệp, biến động nút mạng hoặc phân mảnh mạng là điều bình thường. Với ứng dụng thông thường, chúng chủ yếu gây chậm hoặc mất dữ liệu tạm thời. Với MLS, các hiện tượng đó còn có thể kéo theo hệ quả nghiêm trọng hơn: một số thiết bị đã chuyển sang epoch mới trong khi thiết bị khác vẫn ở epoch cũ, hoặc hai phân vùng mạng cùng tạo ra những Commit khác nhau. Điều này cho thấy lớp mạng P2P chỉ giải quyết việc truyền thông, chứ không tự giải quyết bài toán nhất quán trạng thái mật mã.

2.4.2 go-libp2p và GossipSub

Trong ứng dụng thử nghiệm của đồ án, tầng mạng được xây dựng trên go-libp2p [10]. Đây là thư viện P2P mô-đun hóa, cho phép mỗi thiết bị có một định danh mạng riêng, thiết lập kết nối qua nhiều cơ chế truyền dẫn và trao đổi dữ liệu trên nhiều luồng logic. Những khả năng này phù hợp với yêu cầu của một ứng dụng nhắn tin không phụ thuộc hoàn toàn vào máy chủ trung tâm.

Một thành phần quan trọng của libp2p là GossipSub, tức cơ chế phát tán thông điệp theo mô hình công bố/đăng ký nhận (*publish/subscribe*). Có thể hiểu đơn giản rằng khi một thiết bị gửi thông điệp vào một chủ đề chung, thông điệp đó sẽ được lan truyền dần qua các thiết bị lân cận trong mạng. Cơ chế này phù hợp để phát tán proposal, commit hoặc các thông báo trạng thái trong môi trường P2P.

Tuy nhiên, GossipSub chỉ là cơ chế phát tán thông điệp, không phải cơ chế đồng thuận. Nó không bảo đảm mọi thiết bị nhận thông điệp theo cùng một thứ tự, không tự chọn Commit thắng cuộc khi có xung đột, và cũng không bảo đảm một thông điệp chỉ được nhận đúng một lần. Vì vậy, chỉ đưa MLS lên trên GossipSub là chưa đủ để bảo vệ chuỗi trạng thái của nhóm.

2.4.3 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng

Trong cùng một epoch, nhiều thành viên có thể gửi thông điệp ứng dụng song song. Những thông điệp này không nhất thiết cần một thứ tự mật mã toàn cục như Commit, nhưng giao diện chat vẫn cần một thứ tự hiển thị đủ ổn định giữa các thiết bị. Nếu chỉ dựa vào đồng hồ vật lý, hệ thống dễ gặp sai lệch do lệch giờ hoặc do môi trường mạng không ổn định.

Vì vậy, hệ phân tán thường sử dụng các dạng đồng hồ logic để biểu diễn quan hệ thứ tự giữa các sự kiện. Lamport Clock cho một thứ tự logic đơn giản [11]. Hybrid Logical Clock (HLC) kết hợp thời gian vật lý và bộ đếm logic để tạo ra dấu thời gian gần với thời gian thực hơn, nhưng vẫn ổn định hơn khi đồng hồ giữa các thiết bị không hoàn toàn khớp nhau [12]. Trong đồ án, loại dấu thời gian này chỉ có ý nghĩa đối với việc sắp xếp hiển thị các thông điệp ứng dụng; nó không thay thế epoch, chữ ký hay các cơ chế bảo vệ trạng thái của MLS.

2.4.4 Khoảng trống cần giải quyết

Từ các phân tích trên có thể thấy bài toán của đề tài nằm ở giao điểm giữa mật mã học nhóm và hệ phân tán. MLS cung cấp lõi mật mã mạnh nhưng giả định hệ thống triển khai phải giải quyết việc phân phối và ordering của thông điệp. Libp2p và GossipSub cung cấp hạ tầng truyền thông ngang hàng nhưng không cung cấp thứ tự tuyến tính cho Commit. HLC hỗ trợ sắp xếp thông điệp ứng dụng nhưng không bảo vệ trạng thái mật mã của nhóm [2], [10], [12].

Khoảng trống nghiên cứu vì vậy không nằm ở chỗ thiếu một giao thức mã hóa nhóm, mà nằm ở chỗ thiếu một cơ chế điều phối đủ nhẹ để phù hợp với P2P, nhưng vẫn đủ chặt để giảm nguy cơ rẽ nhánh và hỗ trợ hội tụ trở lại sau khi mạng bất ổn. Khoảng trống đó là cơ sở trực tiếp cho giải pháp sẽ được trình bày ở Chương 3.

Chương 2 đã làm rõ ngữ cảnh lý thuyết của bài toán vận hành MLS trên mạng ngang hàng. Trước hết, chương chỉ ra rằng khó khăn trung tâm của đề tài không chỉ là truyền tin trong môi trường P2P, mà là duy trì một trạng thái mật mã chung cho toàn nhóm khi không còn điểm tuần tự hóa trung tâm. Tiếp theo, chương khảo sát các hướng tiếp cận liên quan, từ Sender Keys, Megolm đến MLS trong mô hình có Delivery Service và các nỗ lực phi tập trung hóa MLS, qua đó làm nổi bật khoảng trống nghiên cứu mà đồ án hướng tới.

Bên cạnh đó, chương cũng trình bày các kiến thức nền tảng cần thiết để hiểu phần phương pháp ở các chương sau, bao gồm epoch, proposal, commit, TreeKEM, các dấu hiệu nhận biết trạng thái và cơ chế gia nhập lại nhóm. Cuối cùng, chương phân tích ngắn gọn các đặc tính của mạng ngang hàng, vai trò của libp2p, GossipSub và đồng hồ logic trong việc hỗ trợ truyền thông và sắp xếp thông điệp ứng dụng. Những nội dung này tạo tiền đề trực tiếp cho Chương 3, nơi giải pháp điều phối phi tập trung của đồ án sẽ được trình bày chi tiết.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

Tổng quan chương.

Chương 2 đã trình bày nền tảng của MLS, TreeKEM, mạng ngang hàng và nguyên nhân dẫn tới phân nhánh trạng thái mật mã khi nhiều Commit cùng xuất hiện trong một Epoch. Trên cơ sở đó, Chương 3 tập trung vào lớp điều phối phi tập trung mà đề án đề xuất để MLS có thể vận hành trong môi trường P2P mà không cần một Delivery Service trung tâm sắp thứ tự Commit.

Trọng tâm của chương này là phương pháp đề xuất ở mức giao thức. Cụ thể, chương lần lượt trình bày bài toán và các yêu cầu thiết kế, cơ chế người ghi duy nhất theo epoch, cơ chế kiểm tra nhất quán epoch, cơ chế phát hiện và hàn gắn phân nhánh, cùng cách dùng đồng hồ logic lai để sắp xếp thông điệp ứng dụng. Ở cuối chương, đề án chỉ dành một mục ngắn để nêu cách ứng dụng thử nghiệm hiện thực các nguyên tắc đó; phần kiểm chứng bằng ứng dụng thử nghiệm và ảnh minh họa được chuyển sang Chương 5 và phụ lục.

3.1 Tổng quan bài toán và hướng tiếp cận

3.1.1 Bài toán cần giải quyết

Trong mô hình MLS thông thường, Delivery Service không chỉ chuyển tiếp thông điệp mà còn giúp các thành viên nhìn thấy cùng một thứ tự Commit [1], [2]. Khi chuyển sang môi trường P2P và bỏ điểm tuần tự hóa trung tâm này, hệ thống phải đối mặt với một rủi ro trực tiếp: hai hoặc nhiều thành viên có thể cùng phát hành Commit cho cùng một Epoch. Khi đó, nhóm không còn tiến hóa theo một chuỗi trạng thái duy nhất, mà có thể tách thành nhiều nhánh trạng thái mật mã khác nhau.

Giả sử nhóm đang ở Epoch E . Nếu thành viên A tạo Commit C_A và thành viên B đồng thời tạo Commit C_B , một phần nút có thể áp dụng C_A trước, trong khi phần còn lại áp dụng C_B trước. Kết quả là nhóm có thể sinh ra hai trạng thái $S_A(E + 1)$ và $S_B(E + 1)$ khác nhau dù cùng mang nhãn Epoch $E + 1$. Khi đó, các nút ở hai nhánh không còn cùng tree hash, cùng lịch sử Commit hay cùng khóa ứng dụng, nên cũng không còn xử lý thông điệp của nhau một cách hợp lệ [5], [6].

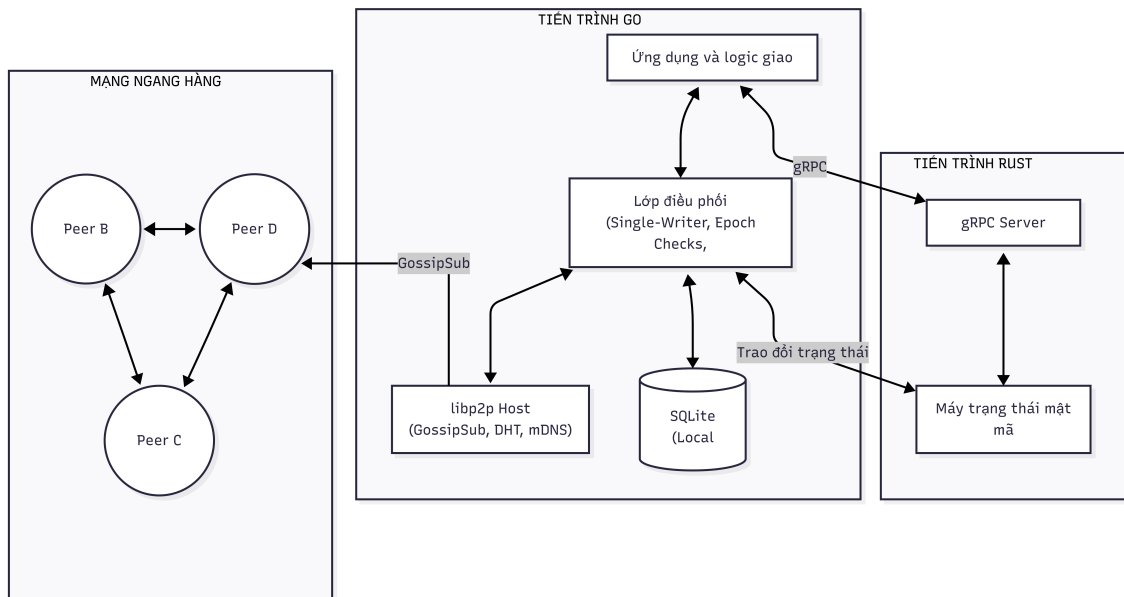
Vì vậy, phương pháp đề xuất trong đề án phải đồng thời đáp ứng bốn yêu cầu: (i) giảm mạnh khả năng xuất hiện concurrent commits khi mạng còn liên thông; (ii) buộc mỗi nút chỉ xử lý thông điệp trên trạng thái MLS tương thích với epoch cục bộ của nó; (iii) phát hiện được sự phân kỳ trạng thái khi network partition xảy ra và hỗ trợ hội tụ trở lại một nhánh hợp lệ; (iv) giữ được một thứ tự hiển thị ổn

định cho các thông điệp ứng dụng trong cùng một Epoch mà không nhằm lẫn vai trò đó với thứ tự Commit.

3.1.2 Kiến trúc logic của giải pháp

Ở mức logic, phương pháp được tổ chức thành hai tầng trách nhiệm. Tầng thứ nhất là tầng điều phối, chịu trách nhiệm quyết định khi nào một nút được quyền tạo Commit, cách phân loại thông điệp theo epoch, cách đối chiếu trạng thái nhánh và cách khôi phục sau phân mảnh mạng. Tầng thứ hai là tầng MLS, chịu trách nhiệm xử lý Proposal, Commit, Welcome, External Join, cũng như mã hóa và giải mã thông điệp theo đúng quy tắc của MLS [1].

Sơ đồ triển khai tổng thể được thể hiện trong Hình 3.1. Ở góc độ phương pháp, hình này cần được đọc như một sơ đồ phân chia trách nhiệm: lớp điều phối không tự cài lại MLS, còn lõi MLS không tự quyết định thứ tự điều phối trong môi trường phân tán. Cách tách này cho phép đồ án giữ nguyên các bảo đảm mật mã của MLS, đồng thời bổ sung lớp điều phối cần thiết cho bối cảnh P2P.



Hình 3.1: Kiến trúc logic của giải pháp và ranh giới trách nhiệm giữa lớp điều phối với lớp MLS

3.1.3 Các cơ chế chính của phương pháp

Phương pháp đề xuất gồm bốn cơ chế gắn chặt với nhau. Cơ chế thứ nhất là người ghi duy nhất theo epoch, nhằm giới hạn quyền phát hành Commit. Cơ chế thứ hai là kiểm tra nhất quán epoch, nhằm ngăn nút xử lý thông điệp trên một trạng thái MLS không tương thích. Cơ chế thứ ba là phát hiện và hàn gắn phân nhánh, nhằm đưa các nút hội tụ trở lại sau network partition. Cơ chế cuối cùng là đồng hồ logic lai HLC, dùng để sắp xếp thông điệp ứng dụng sau khi các thông điệp đó đã đi qua lớp kiểm tra và xử lý của MLS.

Điểm quan trọng là bốn cơ chế này không thay thế MLS. Chúng chỉ tạo ra một lớp bao quanh MLS để bù vào vai trò mà Delivery Service thường đảm nhiệm trong mô hình tập trung. Vì vậy, đóng góp của đề án không nằm ở việc sửa đổi thuật toán mật mã cốt lõi, mà nằm ở cách tổ chức lớp điều phối để MLS vẫn vận hành được trong môi trường P2P.

3.2 Giao thức người ghi duy nhất theo Epoch

3.2.1 Mục tiêu thiết kế

Nguyên nhân trực tiếp của phân nhánh trạng thái trong MLS trên P2P là việc nhiều Commit hợp lệ cùng xuất hiện tại một epoch. Một hướng tiếp cận là chấp nhận cho nhiều Commit xuất hiện rồi xử lý xung đột về sau. Tuy nhiên, cách đó làm tăng độ phức tạp ở tầng trạng thái và khiến hệ thống phải đối mặt với nhiều trạng thái tạm thời hơn [5], [6]. Đề án chọn hướng ngược lại: cố gắng hạn chế xung đột ngay từ đầu bằng cách chỉ cho phép một thành viên phát hành Commit trong mỗi epoch.

Ý tưởng trung tâm của cơ chế này là tách vai trò Proposal và Commit. Thành viên hợp lệ nào cũng có thể phát biểu mong muốn thay đổi nhóm dưới dạng Proposal, nhưng chỉ một thành viên duy nhất trong tập ứng viên hợp lệ mới được quyền gom các Proposal đó để tạo Commit. Thành viên này được gọi là Token Holder của Epoch đang xét.

3.2.2 Tập ứng viên hợp lệ

Token Holder không được chọn từ toàn bộ các nút mà hệ thống nhìn thấy trên mạng, mà chỉ được chọn từ tập ứng viên hợp lệ của nhóm. Tập này được xác định bởi ba loại điều kiện: tư cách thành viên trong trạng thái MLS, tình trạng còn hoạt động trên mạng và chính sách quyền của ứng dụng. Ở mức khái niệm, tập ứng viên tại Epoch E được mô tả như sau:

$$Eligible(E) = \mathcal{M}(E) \cap \mathcal{A}(E) \cap \mathcal{C}(E) \setminus \mathcal{S}(E) \quad (3.1)$$

Trong đó, $\mathcal{M}(E)$ là tập thành viên mà trạng thái MLS còn xem là hợp lệ ở Epoch E ; $\mathcal{A}(E)$ là tập thành viên đang được quan sát là còn hoạt động; $\mathcal{C}(E)$ là tập thành viên mà chính sách ứng dụng cho phép tham gia tạo Commit; còn $\mathcal{S}(E)$ là tập thành viên tạm thời bị loại khỏi vai trò điều phối vì lỗi, ngoại tuyến quá lâu hoặc bị chính sách ứng dụng đình chỉ.

Cách mô hình hóa này giúp phân biệt rõ hai chuyện thường bị trộn lẫn. Một thành viên có thể vẫn còn trong nhóm ở tầng MLS nhưng đang ngoại tuyến, nên không thích hợp giữ vai trò Token Holder. Ngược lại, một nút đang trực tuyến

nhưng không thuộc trạng thái nhóm cũng không có quyền tham gia quyết định Commit. Do đó, chỉ phần giao của các điều kiện trên mới có ý nghĩa đối với bài toán điều phối.

3.2.3 Hàm bầu chọn Token Holder

Sau khi xác định được tập ứng viên hợp lệ, mỗi nút tự tính Token Holder bằng cùng một quy tắc tất định cục bộ. Thành viên có giá trị băm nhỏ nhất khi ghép định danh nhóm, số epoch và định danh nút sẽ được chọn:

$$TH(E) = \operatorname{argmin}_{n \in \text{Eligible}(E)} H(\text{group_id} \parallel E \parallel n) \quad (3.2)$$

Ở đây, H là hàm băm mật mã như SHA-256. Việc đưa group_id vào đầu vào giúp một nút không mặc nhiên giữ ưu thế ở mọi nhóm; việc đưa E vào đầu vào giúp quyền giữ Token thay đổi theo từng epoch; còn định danh nút bảo đảm mọi nút đang có cùng góc nhìn về tập ứng viên sẽ đi tới cùng kết quả. Thuật toán bầu chọn được mô tả trong Thuật toán 1.

Algorithm 1: Thuật toán bầu chọn Token Holder (*ComputeTokenHolder*)

Đầu vào: groupID : định danh nhóm; epoch : Epoch hiện tại; members : tập thành viên MLS; activeView : tập thành viên đang hoạt động; authorized : tập thành viên có quyền tạo Commit; suspended : tập thành viên bị đình chỉ tạm thời.

Đầu ra : holderID : định danh Token Holder được chọn.

$\text{eligible} \leftarrow \text{members} \cap \text{activeView} \cap \text{authorized} \setminus \text{suspended}$;

if eligible là tập rỗng **then**

return Lỗi *NoEligibleCommitter*;

end

$\text{bestID} \leftarrow \text{null}$;

$\text{bestHash} \leftarrow \text{MAX_HASH}$;

for mỗi peerID trong eligible **do**

$\text{data} \leftarrow \text{Encode}(\text{groupID}, \text{epoch}, \text{peerID})$;

$\text{candidateHash} \leftarrow \text{SHA256}(\text{data})$;

if $\text{candidateHash} < \text{bestHash}$ **then**

$\text{bestHash} \leftarrow \text{candidateHash}$;

$\text{bestID} \leftarrow \text{peerID}$;

end

end

return bestID ;

Ưu điểm chính của cách chọn này là không cần thêm một vòng bỏ phiếu mạng cho mỗi Commit. Nếu các nút có cùng trạng thái nhóm và cùng góc nhìn về tập ứng viên, chúng sẽ tự tính ra cùng một Token Holder. Chi phí của bước bầu chọn vì thế chủ yếu là chi phí cục bộ, tăng tuyến tính theo số lượng ứng viên hợp lệ.

3.2.4 Tạo Proposal và Commit

Trong cơ chế đề xuất, Proposal là yêu cầu thay đổi nhóm, còn Commit là thao tác thực sự làm nhóm chuyển sang Epoch mới [1]. Tất cả thành viên hợp lệ đều có thể tạo Proposal. Tuy nhiên, chỉ Token Holder của Epoch hiện tại mới có quyền gom các Proposal hợp lệ để phát hành Commit.

Quy tắc này cho phép hệ thống chấp nhận nhiều ý định thay đổi cùng lúc nhưng vẫn duy trì một điểm chốt duy nhất cho mỗi epoch. Khi mạng còn liên thông, các nút sẽ có cùng Token Holder và vì thế giảm mạnh khả năng sinh concurrent commits. Quy trình tạo Commit của Token Holder được mô tả trong Thuật toán 2.

Algorithm 2: Quy trình tạo Commit của Token Holder
(*CommitByTokenHolder*)

Đầu vào: *localState*: trạng thái MLS cục bộ tại Epoch *E*; *proposalQueue*: hàng đợi Proposal hợp lệ; *holderID*: Token Holder tại Epoch *E*; *selfID*: định danh nút cục bộ.

Đầu ra : Bản tin Commit được phát tán nếu nút cục bộ là Token Holder.

```

if selfID ≠ holderID then
    | Kết thúc;
end
Chọn tập Proposal hợp lệ P từ proposalQueue;
if P rỗng và không có nhu cầu cập nhật cục bộ then
    | Kết thúc;
end
Tạo Commit từ localState và P bằng lớp MLS;
if Commit hợp lệ then
    | Phát tán Commit cho toàn nhóm;
end
else
    | Loại bỏ các Proposal không còn hợp lệ và ghi nhận lỗi;
end

```

Trong nguyên lý này, MLS chịu trách nhiệm kiểm tra tính hợp lệ mật mã của Commit, còn lớp điều phối chịu trách nhiệm quyết định ai được quyền phát hành Commit ở Epoch đó. Việc tách ranh giới như vậy giúp giữ rõ trách nhiệm của từng tầng.

3.2.5 Chuyển quyền và failover

Sau mỗi Commit hợp lệ, nhóm chuyển sang Epoch $E + 1$ và mọi nút tự tính lại Token Holder từ trạng thái mới. Vì quy tắc chọn phụ thuộc vào epoch, quyền phát hành Commit có thể luân chuyển giữa các thành viên thay vì gắn cố định vào một nút.

Trong mạng P2P, Token Holder có thể ngoại tuyến hoặc không kịp phát hành Commit. Để tránh bế tắc, các nút duy trì góc nhìn hoạt động cục bộ của nhóm. Nếu Token Holder hiện tại không còn được quan sát là đang hoạt động hoặc vượt quá thời gian chờ hợp lý, các nút sẽ loại nó khỏi tập ứng viên của Epoch đó và tính lại Token Holder.

Cần nhấn mạnh rằng failover này không phải là một cơ chế đồng thuận Byzantine đầy đủ. Khi network partition xảy ra, các phân vùng khác nhau có thể có các *ActiveView* khác nhau, dẫn tới các Token Holder khác nhau. Do đó, vai trò đúng của Single-Writer là giảm mạnh nguy cơ concurrent commits khi các nút còn có cùng góc nhìn, chứ không phải xóa bỏ hoàn toàn mọi khả năng fork trong mọi điều kiện mạng.

3.3 Kiểm tra nhất quán Epoch

3.3.1 Mục tiêu thiết kế

MLS yêu cầu mỗi Commit và mỗi thông điệp liên quan phải được xử lý trên đúng trạng thái nhóm mà nó thuộc về [1]. Nếu một nút áp dụng bản tin trên một trạng thái MLS không còn tương thích, kết quả có thể là lỗi giải mã, lỗi xác thực hoặc tệ hơn là làm hỏng tiến trình tiến hóa trạng thái cục bộ. Vì vậy, đồ án đặt một lớp kiểm tra ở bên ngoài MLS để phân loại thông điệp trước khi chuyển chúng xuống tầng mật mã.

Trong môi trường P2P bất đồng bộ, một thông điệp có thể đến chậm, đến sớm, bị lặp hoặc đến sai thứ tự. Do đó, lớp điều phối phải trả lời được ba câu hỏi rất cơ bản trước khi xử lý một bản tin: nó thuộc epoch hiện tại, thuộc quá khứ hay đến từ tương lai. Chỉ sau bước phân loại này, nút mới quyết định chuyển bản tin xuống MLS, từ chối, hay đệm lại để đồng bộ hóa trạng thái.

3.3.2 Lớp bao điều phối ở mức khái niệm

Để mô tả quá trình phân loại thông điệp, đồ án dùng một lớp bao điều phối ở mức khái niệm như trong Mã nguồn 3.1. Cần nhấn mạnh rằng đây là mô hình khái niệm của lớp điều phối, dùng để giải thích vai trò của từng trường thông tin trong lập luận. Ứng dụng thử nghiệm có thể mã hóa chi tiết khác đi, miễn là vẫn giữ các ý nghĩa logic tương ứng.

Listing 3.1: Mô hình khái niệm của lớp bao điều phối

```

1  type CoordinationEnvelope struct {
2      GroupID      []byte
3      SenderID     PeerID
4      Type         MessageType
5      Epoch        uint64
6      Timestamp    HLCTimestamp
7      Payload      []byte
8  }

```

Trong mô hình này, *groupID* cho biết bản tin thuộc nhóm nào, *senderID* cho biết nút gửi, *type* cho biết đây là Proposal, Commit hay application message, còn *epoch* cho biết trạng thái MLS mà bản tin được tạo ra từ đó. Trường *timestamp* chỉ có ý nghĩa với việc sắp thứ tự thông điệp ứng dụng; nó không được dùng để quyết định Commit nào hợp lệ.

3.3.3 Phân loại thông điệp theo Epoch

Khi nhận một bản tin, nút cục bộ không xử lý ngay mà trước hết so sánh *env.epoch* với epoch cục bộ của nhóm tương ứng. Có ba tình huống chính. Nếu hai giá trị bằng nhau, bản tin thuộc về trạng thái hiện tại và có thể được xử lý theo luồng bình thường. Nếu *env.epoch* nhỏ hơn epoch cục bộ, bản tin thuộc về quá khứ và không được phép kéo trạng thái nhóm lùi lại. Nếu *env.epoch* lớn hơn epoch cục bộ, nút cục bộ được xem là đang đi sau và phải thực hiện đồng bộ hóa thay vì tăng epoch một cách nhảy cóc.

Thuật toán 3 mô tả bước phân loại cơ bản này.

Algorithm 3: Phân loại bản tin theo epoch (*ProcessIncomingEnvelope*)

Đầu vào: *env*: một bản tin ở tầng điều phối; *localEpoch*: epoch cục bộ của nhóm.

Đầu ra : Hành động xử lý tương ứng.

```

if env.epoch < localEpoch then
    | Từ chối áp dụng bản tin lên trạng thái hiện tại;
else
    | if env.epoch > localEpoch then
    |     | Đưa bản tin vào bộ đệm thông điệp tương lai;
    |     | Kích hoạt quá trình đồng bộ trạng thái;
    | else
    |     | Chuyển bản tin sang luồng xử lý tương ứng với loại của nó;
    | end
end

```

Cách phân loại này giữ cho epoch cục bộ của mỗi nút là một đại lượng đơn điệu không giảm. Nút không tự ý tăng epoch chỉ vì nhìn thấy bản tin từ tương lai, mà chỉ tăng epoch sau khi đã xử lý thành công các bước chuyển trạng thái cần thiết.

3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai

Một thông điệp đến từ tương lai không tự nó là bằng chứng của phân nhánh. Trong nhiều trường hợp, điều đó chỉ cho thấy nút cục bộ đã bỏ lỡ một hoặc nhiều Commit trung gian. Vì vậy, phản ứng đúng của lớp điều phối không phải là kết luận ngay rằng hệ thống đã fork, mà là trước hết thử bắt kịp trạng thái còn thiếu.

Ở mức nguyên lý, quá trình bắt kịp có thể đi theo hai hướng. Hướng thứ nhất là lấy chuỗi Commit còn thiếu và áp dụng tuần tự nếu lịch sử còn đủ để tái hiện. Hướng thứ hai là lấy một ảnh chụp trạng thái nhóm mới hơn rồi gia nhập lại vào trạng thái đó bằng cơ chế hợp lệ của MLS. Cả hai cách đều dựa trên cùng một nguyên tắc: một nút không được nhảy cóc sang trạng thái mới nếu chưa có một con đường hợp lệ để đi từ trạng thái hiện tại của nó sang trạng thái đó.

3.3.5 Xử lý thao tác bị bỏ lỡ

Trong mạng bất đồng bộ, một Proposal do người dùng tạo ra có thể không được đưa vào Commit của Epoch mà nó hướng tới, chẳng hạn vì Token Holder đã chốt một Commit khác trước đó. Đề án xử lý tình huống này theo nguyên tắc đề xuất lại có điều kiện. Cụ thể, nếu sau khi nhóm chuyển sang Epoch mới mà thao tác của người dùng vẫn còn hợp lệ về mặt ngữ nghĩa, nút cục bộ có thể tạo lại Proposal trên trạng thái mới rồi phát tán lại. Nếu thao tác không còn hợp lệ nữa, ứng dụng

phải báo rõ cho người dùng thay vì tiếp tục giữ một ý định đã lỗi thời.

Nguyên tắc này giúp tránh việc đánh đồng giữa một Proposal bị bỏ lỡ và một thao tác phải bị hủy bỏ vĩnh viễn. Đồng thời, nó cũng giữ cho quyền quyết định Commit vẫn nằm ở Token Holder của từng epoch, thay vì cho phép các thao tác cũ tự ý chen vào một trạng thái nhóm đã thay đổi.

3.4 Cơ chế phục hồi phân nhánh trạng thái

3.4.1 Bối cảnh network partition

Single-Writer hoạt động tốt nhất khi các nút còn có cùng góc nhìn về nhóm. Tuy nhiên, trong mạng P2P, network partition có thể chia nhóm thành nhiều phân vùng không liên lạc được với nhau. Khi đó, mỗi phân vùng chỉ quan sát được một phần thành viên đang hoạt động và có thể tính ra các Token Holder khác nhau. Nếu các phân vùng tiếp tục tiến hóa độc lập, hệ thống có thể xuất hiện nhiều nhánh trạng thái MLS khác nhau.

Đồ án không xem đây là một ngoại lệ hiếm có thể bỏ qua, mà xem đó là trường hợp phải được xử lý ngay từ thiết kế. Câu hỏi cần trả lời không phải là làm sao ngăn hoàn toàn mọi phân nhánh trong một môi trường bất đồng bộ, mà là làm sao nhận ra sự phân kỳ đó và đưa hệ thống hội tụ trở lại sau khi các phân vùng nối lại với nhau.

3.4.2 Phát hiện nghi vấn phân nhánh và đối chiếu trạng thái

Để nhận biết tình trạng phân kỳ, mỗi nút định kỳ công bố một bản tóm tắt trạng thái nhánh đang giữ. Ở mức khái niệm, bản tóm tắt này có thể được mô tả như Mã nguồn 3.2.

Listing 3.2: Bản tóm tắt trạng thái nhánh dùng cho đối chiếu

```
1 type GroupStateAnnouncement struct {
2     GroupID          []byte
3     Epoch            uint64
4     TreeHash         []byte
5     LastCommitHash   []byte
6     ActiveMemberCount uint32
7 }
```

Ý nghĩa của thông điệp này không phải là thay thế toàn bộ trạng thái MLS, mà là cung cấp một dấu vết đủ ngắn gọn để lớp điều phối có thể so sánh các nhánh với nhau. Trong đó, epoch cho biết mức tiến hóa của nhánh, tree hash phản ánh trạng thái cây MLS, còn last commit hash đóng vai trò một fingerprint phụ trợ cho lịch sử chuyển trạng thái gần nhất.

Tuy nhiên, một announcement khác với trạng thái cục bộ chưa đủ để kết luận ngay rằng phải hàn gắn phân nhánh. Nếu phía từ xa đang ở epoch cao hơn, khả năng đầu tiên cần kiểm tra là nút cục bộ chỉ đang đi sau và chưa kịp đồng bộ. Vì vậy, đề án tách quá trình này thành ba bước. Bước thứ nhất là phát hiện nghi vấn phân nhánh từ bản tóm tắt trạng thái. Bước thứ hai là thử bắt kịp trạng thái còn thiếu nếu dấu hiệu chủ yếu là chênh lệch epoch. Bước thứ ba là chỉ kết luận có phân nhánh thực sự khi sự khác biệt vẫn tồn tại sau bước đối chiếu hoặc khi hai nút cùng ở một epoch nhưng tree hash hay dấu vết Commit không còn khớp nhau.

Cách tổ chức này giúp tránh một ngộ nhận dễ gặp: khác trạng thái tạm thời do chưa đồng bộ không đồng nghĩa ngay với phân nhánh thật. Do đó, announcement nên được xem là công cụ phát hiện sớm và đối chiếu trạng thái, chứ không phải một bằng chứng tuyệt đối đứng một mình trong mọi bối cảnh mạng.

3.4.3 Hàm trọng số nhánh

Khi hệ thống đã xác định rằng đang tồn tại nhiều nhánh khác nhau của cùng một nhóm, nó cần một quy tắc tất định để chọn nhánh tiếp tục. Đề án dùng một hàm trọng số nhánh với thứ tự ưu tiên từ trái sang phải như sau:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (3.3)$$

Trong đó, $C_{active}(B)$ là số thành viên đang hoạt động quan sát được trên nhánh B , $E(B)$ là epoch hiện tại của nhánh, còn $H_{commit}(B)$ là giá trị băm của Commit gần nhất gắn với nhánh đó. Các nhánh được so sánh theo thứ tự từ điển. Nhánh có nhiều thành viên hoạt động hơn được ưu tiên trước; nếu bằng nhau, nhánh có epoch cao hơn được ưu tiên; nếu vẫn bằng nhau, giá trị băm Commit đóng vai trò phá hòa tất định.

Thuật toán 4 mô tả quy tắc so sánh này.

Algorithm 4: So sánh trọng số nhánh (*CompareBranchWeight*)

Đầu vào: B_a, B_b : hai nhánh cần so sánh.

Đầu ra : Nhánh được chọn làm nhánh tiếp tục.

if $C_{active}(B_a) \neq C_{active}(B_b)$ **then**

Chọn nhánh có C_{active} lớn hơn;
Kết thúc;

end

if $E(B_a) \neq E(B_b)$ **then**

Chọn nhánh có epoch lớn hơn;
Kết thúc;

end

Chọn nhánh có H_{commit} lớn hơn theo thứ tự từ điển;

Quy tắc này không khẳng định rằng nhánh được chọn là nhánh “đúng” theo một nghĩa tuyệt đối ngoài hệ thống. Vai trò của nó khiêm tốn hơn: bảo đảm rằng nếu các nút cuối cùng quan sát được cùng tập nhánh và cùng áp dụng một quy tắc tất định, chúng sẽ hội tụ về cùng một quyết định phục hồi.

3.4.4 Quy trình hàn gắn phân nhánh

Sau khi một nhánh thắng đã được chọn, hệ thống không cố gắng hợp nhất trực tiếp hai trạng thái MLS đã phân kỳ. Việc trộn khóa, trộn transcript hay ghép thủ công hai trạng thái như vậy nằm ngoài mô hình vận hành hợp lệ của MLS. Thay vào đó, nút ở nhánh thua sẽ lấy thông tin cần thiết của nhánh thắng rồi gia nhập lại bằng cơ chế hợp lệ của MLS, cụ thể là External Join [1].

Quy trình tổng quát được mô tả trong Thuật toán 5.

Algorithm 5: Quy trình hàn gắn phân nhánh (*ForkHealing*)

Đầu vào: *Branches*: tập các nhánh đang được quan sát sau khi đã đối chiếu trạng thái.

Đầu ra : Một trạng thái MLS hợp lệ chung để nhóm tiếp tục vận hành.

$winningBranch \leftarrow SelectBranch(Branches);$

for mỗi nút đang ở nhánh thua **do**

Lấy thông tin nhóm từ *winningBranch*;
Gia nhập lại bằng cơ chế *ExternalJoin*;
Hủy bỏ trạng thái MLS cũ của nhánh thua;

end

Nếu chính sách ứng dụng cho phép phục hồi thông điệp đã phát sinh ở nhánh

thua, nút chỉ được phát lại các thông điệp do chính nó tạo ra và phải mã hóa lại chúng dưới trạng thái MLS mới. Quy tắc này giữ cho quá trình phục hồi dữ liệu ứng dụng không xâm phạm ranh giới trách nhiệm của MLS và cũng không cho phép một nút tự ý đại diện cho thông điệp của nút khác.

3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock

3.5.1 Lý do cần HLC

Epoch trong MLS chỉ quyết định thứ tự thay đổi trạng thái mật mã. Nó không tạo ra một thứ tự toàn phần cho các thông điệp ứng dụng được gửi gần đồng thời trong cùng một Epoch. Trong môi trường P2P, hai thiết bị có thể nhận các thông điệp đó theo thứ tự khác nhau do độ trễ mạng, dù cả hai cuối cùng đều giải mã được đầy đủ.

Nếu giao diện chỉ hiển thị theo thời điểm nhận cục bộ, hai người dùng có thể nhìn thấy lịch sử hội thoại theo hai thứ tự khác nhau. Để giảm tình huống đó, đồ án dùng đồng hồ logic lai HLC. HLC kết hợp thành phần thời gian gần với đồng hồ vật lý với một bộ đếm logic, từ đó tạo ra một khóa sắp thứ tự ổn định hơn cho các thông điệp ứng dụng [12].

$$HLC = (L, C, N) \quad (3.4)$$

Trong đó, L là thành phần thời gian logic bám theo thời gian vật lý, C là bộ đếm logic dùng khi nhiều sự kiện có cùng L , còn N là định danh nút dùng để phá hòa một cách tất định.

3.5.2 Tạo timestamp khi gửi tin

Khi gửi một thông điệp ứng dụng, nút cục bộ lấy thời gian vật lý hiện tại rồi so sánh với giá trị L đang lưu. Nếu thời gian vật lý đã tiến lên, L được cập nhật theo thời gian vật lý đó và bộ đếm C trở về 0. Nếu thời gian vật lý chưa tăng, hệ thống chỉ tăng bộ đếm logic. Thuật toán này được mô tả trong Thuật toán 6.

Algorithm 6: Sinh thời gian logic lai (HLC_Now)

Đầu vào: hlc : trạng thái HLC cục bộ; $nodeID$: định danh nút cục bộ.

Đầu ra : Timestamp HLC mới ($L, C, nodeID$).

$pt \leftarrow$ thời gian vật lý hiện tại;

if $pt > hlc.L$ **then**

$hlc.L \leftarrow pt$;

$hlc.C \leftarrow 0$;

end

else

$hlc.C \leftarrow hlc.C + 1$;

end

return ($hlc.L, hlc.C, nodeID$);

3.5.3 Cập nhật HLC khi nhận tin

Khi nhận một thông điệp ứng dụng mang timestamp HLC, nút cục bộ cập nhật đồng hồ của mình theo giá trị lớn nhất giữa đồng hồ cục bộ, timestamp nhận được và thời gian vật lý hiện tại. Cách cập nhật này bảo toàn thứ tự nhân quả cơ bản mà không buộc các thiết bị phải đồng bộ đồng hồ tuyệt đối. Thuật toán tương ứng được mô tả trong Thuật toán 7.

Algorithm 7: Cập nhật đồng hồ logic lai (HLC_Update)

Đầu vào: hlc : trạng thái HLC cục bộ; msg : timestamp HLC nhận được
 $(msg.L, msg.C, msg.NodeID)$.

Đầu ra : Trạng thái HLC cục bộ sau khi cập nhật.

```

 $pt \leftarrow$  thời gian vật lý hiện tại;
 $newL \leftarrow \max(hlc.L, msg.L, pt)$ ;
if  $newL == hlc.L$  and  $newL == msg.L$  then
    |  $hlc.C \leftarrow \max(hlc.C, msg.C) + 1$ ;
end
else if  $newL == hlc.L$  then
    |  $hlc.C \leftarrow hlc.C + 1$ ;
end
else if  $newL == msg.L$  then
    |  $hlc.C \leftarrow msg.C + 1$ ;
end
else
    |  $hlc.C \leftarrow 0$ ;
end
 $hlc.L \leftarrow newL$ ;
return  $hlc$ ;

```

3.5.4 Vai trò của HLC trong hệ thống

HLC chỉ được dùng để sắp thứ tự hiển thị các thông điệp ứng dụng đã vượt qua bước kiểm tra epoch và đã được MLS xử lý hợp lệ. Nó không quyết định Commit nào đúng, không thay thế lớp điều phối ở tầng trạng thái và không được phép ảnh hưởng tới key schedule của MLS. Sự tách vai trò này là điều kiện cần để hệ thống vừa giữ đúng mô hình bảo mật của MLS, vừa có một thứ tự hiển thị ổn định hơn ở tầng ứng dụng.

3.6 Hiện thực ứng dụng thử nghiệm phục vụ kiểm chứng

3.6.1 Ranh giới trách nhiệm trong ứng dụng thử nghiệm

Ở mức hiện thực ứng dụng thử nghiệm, các nguyên tắc trên được ánh xạ thành một kiến trúc hai tầng gồm một tiến trình chủ chịu trách nhiệm điều phối và một thành phần MLS chịu trách nhiệm xử lý mật mã. Trạng thái bền vững của nhóm được lưu ở phía host; khi cần xử lý Proposal, Commit, External Join hoặc application message, host lấy trạng thái hiện tại, chuyển nó cho tầng MLS xử lý rồi ghi lại trạng thái mới sau khi thao tác hoàn tất.

Cách hiện thực này có ý nghĩa chủ yếu ở mặt phương pháp. Nó cho phép ứng dụng thử nghiệm giữ đúng ranh giới mà chương này đã nêu: quyết định điều phối nằm ở tầng ngoài, còn MLS chỉ xử lý các phép biến đổi trạng thái hợp lệ của chính nó. Vì vậy, việc ứng dụng này dùng công nghệ nào ở tầng giao diện không làm thay đổi bản chất của đóng góp nghiên cứu.

3.6.2 Vai trò của ứng dụng desktop

Trong ứng dụng thử nghiệm của đồ án, ứng dụng desktop không được xem là phần đóng góp thuật toán mới. Vai trò của nó là hiện thực hóa các luồng thao tác quan trọng nhất để kiểm chứng phương pháp: tham gia hệ thống bằng bundle, tạo và quản trị nhóm, và trao đổi thông điệp trên nhiều nút.

Vì lý do đó, các màn hình cụ thể, ảnh chụp ứng dụng và kịch bản demo không được trình bày chi tiết ngay trong Chương 3. Thay vào đó, Chương 5 sẽ dùng ứng dụng thử nghiệm như một môi trường kiểm chứng đầu cuối, còn phụ lục sẽ tập hợp đầy đủ các hình minh họa cần thiết cho việc đối chiếu.

3.7 Tính đúng đắn trực giác của phương pháp

Trong điều kiện các nút còn có cùng góc nhìn gần đúng về nhóm, Single-Writer giúp giảm mạnh khả năng xuất hiện concurrent commits vì mỗi Epoch chỉ có một Token Holder được quyền phát hành Commit. Kết hợp với lớp kiểm tra epoch, cơ chế này buộc mọi bước tiến hóa của trạng thái cục bộ phải đi qua một chuỗi Commit hợp lệ thay vì cho phép các nút tự ý xử lý bản tin trên các trạng thái không tương thích.

Khi network partition xảy ra, phương pháp không giả định rằng hệ thống vẫn giữ được một trạng thái duy nhất tại mọi thời điểm. Thay vào đó, nó chấp nhận khả năng phân kỳ tạm thời, nhưng yêu cầu lớp điều phối phải nhận ra sự khác biệt giữa các nhánh, thử bắt kịp trạng thái khi cần, rồi cuối cùng chọn một nhánh để hội tụ trở lại bằng cơ chế hợp lệ của MLS. Điểm cốt lõi ở đây là hệ thống không cố gắng hợp nhất trực tiếp hai trạng thái MLS đã rẽ nhánh.

Cuối cùng, HLC giúp giữ một thứ tự hiển thị ổn định hơn cho các thông điệp ứng dụng sau khi các thông điệp đó đã được xử lý hợp lệ. Nhờ vậy, phương pháp đề xuất không chỉ quan tâm tới nhất quán trạng thái mật mã, mà còn quan tâm tới tính quan sát được của hệ thống ở tầng ứng dụng, nhưng vẫn giữ rõ ranh giới giữa hai loại ordering.

Kết chương. Chương 3 đã trình bày lớp điều phối phi tập trung mà đồ án đề xuất cho MLS trên mạng ngang hàng. Phần cốt lõi của phương pháp gồm bốn cơ chế: người ghi duy nhất theo epoch để hạn chế concurrent commits, kiểm tra nhất quán

epoch để bảo vệ trạng thái cục bộ, phát hiện và hàn gắn phân nhánh để hỗ trợ hội tụ sau network partition, và HLC để sắp thứ tự thông điệp ứng dụng.

Các cơ chế trên tạo thành phần nghiên cứu trung tâm của đề án. Ứng dụng thử nghiệm chỉ đóng vai trò hiện thực hóa và kiểm chứng các nguyên tắc đó trong một hệ thống chạy được. Trên cơ sở này, Chương 4 sẽ tiếp tục phân tích các bất biến và giới hạn lý thuyết của phương pháp trước khi Chương 5 chuyển sang phần kiểm chứng bằng ứng dụng thử nghiệm và thực nghiệm.

CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT

Tổng quan chương.

Chương 3 đã trình bày lớp điều phối phi tập trung được đề xuất để đưa MLS vào môi trường mạng ngang hàng. Tuy nhiên, việc nêu cơ chế thôi chưa đủ để khẳng định hướng tiếp cận là hợp lý. Vì vậy, chương này tiếp tục phân tích cơ sở lý thuyết của các cơ chế đã nêu, làm rõ chúng bảo toàn được những bất biến nào, trong phạm vi giả định nào và giới hạn của chúng nằm ở đâu.

Trong phần còn lại, chương sẽ lần lượt phân tích tính đơn điệu của epoch cục bộ, tính đơn trị của Token Holder khi các nút có cùng góc nhìn, khả năng nhận biết phân nhánh trạng thái, lý do không gộp trực tiếp hai trạng thái MLS đã tách nhánh, vai trò giới hạn của HLC và chi phí lý thuyết của lớp điều phối. Toàn bộ lập luận trong chương chỉ bám vào mô hình giao thức đã nêu ở Chương 3, không dựa vào chi tiết giao diện hay kỹ thuật hiện thực của ứng dụng thử nghiệm.

4.1 Mô hình phân tích và giả định

Xét một nhóm MLS có định danh *groupID* và trạng thái hiện hành ở epoch *E*. Mỗi nút giữ một trạng thái cục bộ, trong đó phần lõi là trạng thái MLS dùng để xử lý Proposal, Commit và application message; phần còn lại là metadata điều phối phục vụ bầu chọn Token Holder, đối chiếu trạng thái và phát hiện phân nhánh. Trong phạm vi chương này, cần phân biệt rõ hai lớp thông tin đó. Trạng thái MLS đầy đủ quyết định tính hợp lệ về mật mã, còn metadata điều phối chỉ đóng vai trò quan sát và điều hướng quá trình hội tụ.

Mạng ngang hàng được xem là môi trường bất đồng bộ. Thông điệp có thể đến trễ, đến sai thứ tự, bị lặp hoặc tạm thời không đến được. Khi mạng còn liên thông, các nút cuối cùng có thể trao đổi lại thông tin với nhau, nhưng không giả định rằng mọi nút quan sát thông điệp theo cùng một thứ tự. Khi xảy ra phân vùng mạng, hai tập nút khác nhau có thể hình thành hai góc nhìn khác nhau về những thành viên còn đang hoạt động.

Ký hiệu $ActiveView_i(E)$ là tập thành viên mà nút *i* quan sát là còn hoạt động tại epoch *E*. Trong điều kiện ổn định, các nút có xu hướng hội tụ về cùng một *ActiveView*. Khi mạng bị chia cắt, hai phân vùng có thể giữ hai *ActiveView* khác nhau. Chính khác biệt này làm cho cùng một quy tắc bầu chọn tất định vẫn có thể cho ra hai Token Holder khác nhau ở hai phân vùng khác nhau.

Các lập luận dưới đây được xây dựng trong phạm vi giả định của đề án: các nút nhìn chung tuân thủ giao thức, các thao tác MLS cốt lõi được OpenMLS xử lý

đúng, và mục tiêu chính là phân tích hành vi của lớp điều phối khi thiếu Delivery Service tập trung. Chương này không tìm cách bao phủ toàn bộ không gian tấn công của một hệ phân tán đối kháng mạnh.

4.2 Bất biến 1: Epoch cục bộ chỉ được giữ nguyên hoặc tăng

4.2.1 Phát biểu bất biến

Tại mỗi nút, epoch cục bộ chỉ được giữ nguyên hoặc tăng lên sau khi nút xử lý thành công một Commit hợp lệ. Nút không được áp dụng một thông điệp thuộc epoch cũ lên trạng thái hiện tại, vì điều đó có thể kéo trạng thái nhóm quay về một mốc cũ và làm cho bản mã bị xử lý trên một trạng thái MLS không còn tương thích.

Bất biến này có thể viết ngắn gọn như sau:

$$E_i(t+1) \geq E_i(t)$$

trong đó $E_i(t)$ là epoch cục bộ của nút i tại thời điểm logic t .

4.2.2 Cơ sở từ MLS

Trong MLS, mỗi Commit hợp lệ đưa nhóm sang một epoch mới và làm thay đổi trạng thái cây, lịch sử bắt tay cùng lịch trình khóa của nhóm [1]. Vì vậy, một thông điệp được tạo ở epoch cũ không thể được xem là tương thích một cách mặc nhiên với trạng thái hiện tại. Nếu nút cho phép áp dụng tùy tiện các thông điệp cũ, chuỗi tiến hóa của trạng thái nhóm sẽ bị phá vỡ.

4.2.3 Cơ chế bảo toàn bất biến

Chương 3 đã đề xuất cơ chế kiểm tra epoch ở lớp điều phối trước khi chuyển thông điệp xuống MLS. Quy tắc xử lý có thể tóm tắt như Bảng 4.1.

Quan hệ epoch	Hành động của lớp điều phối
$E_{msg} = E_{local}$	Thông điệp được chuyển tiếp cho lớp MLS xử lý theo luồng tiến hóa bình thường.
$E_{msg} < E_{local}$	Thông điệp không được áp dụng lên trạng thái hiện tại, nhằm tránh kéo trạng thái cục bộ lùi về một mốc cũ.
$E_{msg} > E_{local}$	Thông điệp chưa được xử lý ngay; nút tạm giữ lại và kích hoạt bước đồng bộ trạng thái.

Bảng 4.1: Quy tắc xử lý thông điệp theo epoch

Mệnh đề 1. Nếu một nút chỉ tăng epoch sau khi xử lý thành công Commit hợp lệ và không áp dụng thông điệp thuộc epoch cũ lên trạng thái hiện tại, thì epoch cục bộ của nút là đơn điệu không giảm.

Lập luận. Theo Bảng 4.1, mọi thông điệp cũ đều bị chặn ở lớp điều phối, nên trạng thái MLS hiện tại không bị đề ngược bởi dữ liệu quá khứ. Đối với thông điệp đi trước trạng thái cục bộ, nút không áp dụng ngay mà phải đồng bộ lại chuỗi trạng thái. Bởi trạng thái mới chỉ xuất hiện sau khi xử lý Commit hợp lệ, chuỗi giá trị epoch tại một nút không thể tự giảm xuống.

Bất biến này không đồng nghĩa với việc mọi nút luôn giữ cùng một epoch ở mọi thời điểm. Trong mạng ngang hàng, có nút có thể tạm thời chậm hơn do mất kết nối hoặc nhận thông điệp muộn. Điều bất biến bảo toàn là mỗi nút không tự kéo trạng thái của chính mình lùi về phía sau.

4.3 Bất biến 2: Với cùng một view, chỉ có một Token Holder

4.3.1 Phát biểu bất biến

Ở epoch E , nếu hai nút có cùng $groupID$, cùng số epoch và cùng tập ứng viên hợp lệ, chúng phải tính ra cùng một Token Holder. Token Holder là nút duy nhất được quyền tạo Commit trong view đó.

Tập ứng viên hợp lệ được xác định như sau:

$$\begin{aligned} Eligible(E) = & GroupMembers(E) \cap ActiveView(E) \\ & \cap AuthorizedCommitters(E) \setminus RemovedOrSuspended(E) \end{aligned} \quad (4.1)$$

Trong đó, $GroupMembers(E)$ là tập thành viên còn có mặt trong trạng thái MLS tại epoch E ; $ActiveView(E)$ là tập thành viên còn được quan sát là hoạt động; $AuthorizedCommitters(E)$ là tập thành viên được chính sách ứng dụng cho phép tạo Commit; và $RemovedOrSuspended(E)$ là tập thành viên bị loại tạm thời khỏi vòng bầu chọn.

Token Holder được tính bằng quy tắc tất định:

$$TokenHolder(E) = \operatorname{argmin}_{n \in Eligible(E)} H(groupID \parallel E \parallel n) \quad (4.2)$$

trong đó H là hàm băm mật mã, chẳng hạn SHA-256.

Mệnh đề 2. Nếu hai nút có cùng $groupID$, cùng epoch E và cùng tập $Eligible(E)$, thì chúng chọn cùng một Token Holder.

Lập luận. Khi đầu vào của Công thức (4.2) là như nhau, tập giá trị băm sinh ra trên hai nút cũng như nhau. Do phép chọn argmin là tất định, kết quả cuối cùng bắt buộc trùng nhau. Nói cách khác, hệ thống không cần thêm một vòng bỏ phiếu

mạng chỉ để quyết định ai là người tạo Commit trong cùng một view.

4.3.2 Ý nghĩa đối với concurrent commits

Trong MLS, Commit là thao tác làm thay đổi trạng thái nhóm và đưa nhóm sang epoch mới [1]. Nếu mọi thành viên đều có thể tự Commit tại cùng một epoch, hệ thống sẽ dễ sinh nhiều nhánh cạnh tranh. Cơ chế Single-Writer giải quyết đúng điểm này bằng cách giới hạn quyền tạo Commit vào một nút duy nhất trong từng view.

Điều cần nhấn mạnh là bất biến trên chỉ đúng khi các nút đang có cùng góc nhìn về nhóm. Nếu hai phân vùng giữ hai *ActiveView* khác nhau, chúng có thể hình thành hai tập $Eligible(E)$ khác nhau và từ đó chọn ra hai Token Holder khác nhau. Vì vậy, Single-Writer không phải là lời hứa rằng hệ thống không bao giờ phân nhánh; vai trò đúng của nó là giảm mạnh khả năng sinh concurrent commits khi mạng còn đủ liên thông để các nút chia sẻ cùng một view.

4.4 Bất biến 3: Phân nhánh phải được nhận biết sau bước đối chiếu lịch sử

4.4.1 Phân biệt chậm đồng bộ và phân nhánh thực sự

Trong mạng ngang hàng, khác biệt quan sát được giữa hai nút chưa đủ để kết luận rằng nhóm đã phân nhánh. Nếu một nút có epoch thấp hơn nút khác, khả năng tự nhiên hơn là nó chỉ đang chậm đồng bộ. Ngay cả khi hai nút đều công bố những fingerprint trạng thái khác nhau, kết luận có fork vẫn chỉ nên được đưa ra sau khi đã kiểm tra xem một bên có còn là phần kéo dài hợp lệ của bên kia hay không. Vì vậy, bài toán ở đây không đơn thuần là so sánh hai trạng thái hiện tại, mà là đối chiếu lịch sử Commit mà mỗi nút đã chấp nhận.

4.4.2 Cam kết cho tiền tổ lịch sử Commit

Ký hiệu $C_i(e)$ là giá trị băm của Commit mà nút i đã chấp nhận tại epoch e . Từ dãy các Commit này, mỗi nút xây dựng một giá trị cam kết cho toàn bộ tiền tổ lịch sử:

$$R_i(0) = r_0$$

$$R_i(e) = H(R_i(e-1) \parallel C_i(e))$$

trong đó r_0 là hằng khởi tạo chung, còn H là hàm băm mật mã. Giá trị $R_i(e)$ không chỉ đại diện cho Commit gần nhất, mà ràng buộc toàn bộ chuỗi Commit từ epoch 1 đến epoch e . Nếu hai nút có cùng $R(e)$ tại cùng một mốc e , thì trong phạm vi giả định rằng các Commit được chuẩn hóa và được băm theo cùng một quy tắc,

chúng đã đi qua cùng một tiền tố lịch sử đến mốc đó.

Để phục vụ bước đối chiếu, mỗi nút công bố định kỳ một bản tóm tắt trạng thái ngắn gọn:

$$StateSummary = (groupID, E, treeHash, \\ historyHash)$$

Trong đó, *treeHash* phản ánh trạng thái cây MLS hiện tại, còn *historyHash* chính là $R(E)$. Bản tóm tắt này không thay thế trạng thái MLS đầy đủ, nhưng đủ để làm tín hiệu ban đầu cho bước đối chiếu. Nếu hai nút đang ở cùng epoch, việc so sánh trực tiếp hai giá trị $R(E)$ là đủ để biết chúng có cùng lịch sử Commit đến epoch đó hay không. Nếu một nút A đang ở epoch E_A thấp hơn nút B ở epoch E_B , nút A chưa thể kết luận gì chỉ từ $R_B(E_B)$. Khi đó, A cần thêm một lượt hỏi–đáp để lấy đúng giá trị $R_B(E_A)$ rồi mới đối chiếu với $R_A(E_A)$.

Mệnh đề 3. Trong phạm vi giả định của đề án, giả sử hai nút A và B thuộc cùng một nhóm và $E_A \leq E_B$. Sau khi A nhận được *StateSummary* của B và, nếu cần, lấy thêm giá trị $R_B(E_A)$ qua một lượt đối chiếu bổ sung, nếu

$$R_B(E_A) \neq R_A(E_A)$$

thì lớp điều phối có cơ sở để kết luận rằng A và B không còn đứng trên cùng một chuỗi Commit. Ngược lại, nếu

$$R_B(E_A) = R_A(E_A)$$

thì khác biệt quan sát được vẫn còn có thể giải thích bằng việc A đang chậm đồng bộ và lịch sử của A chỉ là một tiền tố của lịch sử ở B .

Lập luận. Do $R_i(e)$ được xây dựng bằng cách băm lũy tiến trên toàn bộ dãy Commit từ 1 đến e , nên điều kiện $R_B(E_A) = R_A(E_A)$ có nghĩa là hai nút đã chấp nhận cùng một tiền tố lịch sử đến epoch E_A . Nếu B còn đi xa hơn sau mốc đó, thì sự khác biệt hiện tại chỉ phản ánh việc B đã xử lý thêm các Commit mới mà A chưa kịp nhận. Ngược lại, nếu $R_B(E_A) \neq R_A(E_A)$, thì phải tồn tại ít nhất một epoch không vượt quá E_A mà hai bên đã chấp nhận hai Commit khác nhau, hoặc đã đi qua hai tiền tố lịch sử khác nhau. Khi đó, khác biệt quan sát được không còn là chậm đồng bộ đơn thuần mà đã trở thành khác biệt về lineage của trạng thái nhóm.

Trong lập luận này, *treeHash* vẫn giữ vai trò một fingerprint ngắn gọn của trạng thái MLS hiện hành và vẫn hữu ích trong bước đối chiếu trạng thái hoặc giai đoạn phục hồi sau phân nhánh. Tuy nhiên, tín hiệu quyết định để phân biệt giữa “đi sau trên cùng một nhánh” và “đã tách sang nhánh khác” là phép kiểm tra tính trùng khớp của tiền tố lịch sử Commit tại epoch nhỏ hơn, chứ không phải chỉ là so sánh Commit gần nhất ở đỉnh hiện tại.

4.5 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh

4.5.1 Vì sao không thể gộp trực tiếp hai nhánh

Sau khi phân nhánh, mỗi nhánh MLS có lịch sử Commit, trạng thái cây, lịch sử bắt tay và lịch trình khóa riêng. Nếu lớp điều phối tự ý trộn hai trạng thái đó để tạo ra một trạng thái mới, trạng thái mới sẽ không còn là kết quả của một chuỗi thao tác MLS hợp lệ [1]. Khi ấy, hệ thống không còn có thể dựa vào các bảo đảm của MLS/OpenMLS để khẳng định trạng thái mới là hợp lệ về mặt mã.

Vì lý do đó, phương pháp của đề án không tìm cách hợp nhất trực tiếp hai nhánh đã tách. Thay vào đó, nó chọn một nhánh tiếp tục và đưa các nút ở nhánh còn lại gia nhập lại nhóm theo con đường MLS hợp lệ. Cách xử lý này giữ cho quá trình hội tụ vẫn nằm trong mô hình vận hành chuẩn của MLS.

4.5.2 Quy tắc chọn nhánh

Giả sử sau khi mạng nối lại, hệ thống quan sát được tập các nhánh

$$\mathcal{B} = \{B_1, B_2, \dots, B_k\}$$

Mỗi nhánh B được gắn với một trọng số:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (4.3)$$

Trong đó, $C_{active}(B)$ là số thành viên còn hoạt động của nhánh, $E(B)$ là epoch hiện tại của nhánh, còn $H_{commit}(B)$ là giá trị băm của Commit gần nhất dùng làm khóa phá hòa. Hai nhánh được so sánh theo thứ tự từ điển trên ba thành phần này.

Quy tắc trên không nhằm khẳng định nhánh thắng là nhánh "đúng tuyệt đối" trong mọi trạng thái của mạng. Vai trò của nó khi thông tin đã đủ là tạo ra một quy tắc hội tụ nhất quán: nhiều nút quan sát cùng một tập nhánh sẽ có xu hướng chọn cùng một nhánh tiếp tục.

4.5.3 External Join như cơ chế hội tụ

Sau khi một nhánh được chọn, các nút ở nhánh thua không mang trạng thái cũ đi ghép với nhánh thắng. Chúng phải gia nhập lại bằng một thao tác hợp lệ của MLS, chẳng hạn External Join hoặc một luồng tương đương tùy theo chính sách hiện thực. Khi đó, các khóa cũ của nhánh thua phải bị loại bỏ, còn trạng thái mới của nút được xây dựng lại từ nhánh đã được chọn.

Mệnh đề 4. Nếu các nút quan sát cùng một tập nhánh và cùng áp dụng quy tắc chọn nhánh ở Công thức (4.3), thì chúng sẽ có cùng cơ sở để chọn nhánh tiếp tục; việc đưa các nút của nhánh thua gia nhập lại qua thao tác hợp lệ của MLS giúp hệ thống hội tụ mà không phải tự chế ra một trạng thái mật mã mới.

Lập luận. Thành phần $C_{active}(B)$ ưu tiên nhánh còn duy trì được nhiều thành viên hoạt động hơn, $E(B)$ ưu tiên nhánh tiến xa hơn khi số thành viên còn hoạt động bằng nhau, còn $H_{commit}(B)$ đóng vai trò phá hòa tất định nếu hai tiêu chí trước vẫn trùng nhau. Sau khi đã chọn được nhánh tiếp tục, quá trình gia nhập lại không dựa trên việc gộp hai trạng thái cũ, mà dựa trên chính các cơ chế thay đổi thành viên mà MLS cho phép. Nhờ vậy, đường hội tụ vẫn nằm trong không gian trạng thái hợp lệ của giao thức.

4.6 Xử lý thông điệp phát sinh ở nhánh thua

Trong thời gian phân vùng mạng, người dùng ở mỗi nhánh vẫn có thể gửi application message. Khi hệ thống hội tụ trở lại, các thông điệp sinh ra ở nhánh thua đặt ra một câu hỏi riêng: nội dung đó có thể còn giá trị với người dùng, nhưng bản mã gốc của nó lại thuộc về một trạng thái MLS không còn tiếp tục được sử dụng.

Phương pháp của đồ án không phát lại nguyên trạng các bản mã này. Nếu chính sách ứng dụng cho phép phục hồi nội dung, mỗi nút chỉ được tự mã hóa lại các thông điệp do chính nó tạo ra, rồi gửi chúng như các application message mới dưới trạng thái hợp lệ của nhánh thắng. Cách làm này giữ ba ranh giới quan trọng. Thứ nhất, hệ thống không kéo dài vòng đời sử dụng của các khóa thuộc nhánh thua. Thứ hai, một nút không được tự ý phát lại nội dung thay cho nút khác. Thứ ba, thông điệp sau khi quay lại hệ thống sẽ là thông điệp mới của trạng thái hiện hành, chứ không phải là mảnh ghép thô của trạng thái cũ.

4.7 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit

4.7.1 Phân biệt hai loại thứ tự

Trong hệ thống cần tách bạch hai loại thứ tự. Loại thứ nhất là thứ tự Commit, quyết định chuỗi tiến hóa của trạng thái MLS. Loại thứ hai là thứ tự hiển thị application message sau khi chúng đã được chấp nhận và giải mã hợp lệ. Hai loại thứ tự này phục vụ hai mục tiêu khác nhau nên không thể dùng lẫn cho nhau.

MLS đã có cơ chế riêng để ràng buộc application message với đúng nhóm, đúng epoch và đúng ngữ cảnh mật mã [1]. Tuy nhiên, ở mức ứng dụng, các nút vẫn cần một quy tắc ổn định để hiển thị cùng một tập thông điệp theo thứ tự nhất quán hơn. HLC được đưa vào đúng cho mục đích đó.

4.7.2 Cơ sở của HLC

Timestamp HLC của một thông điệp được biểu diễn dưới dạng

$$HLC = (L, C, NodeID) \quad (4.4)$$

trong đó L là thành phần thời gian logic gắn với thời gian vật lý, C là bộ đếm logic, còn $NodeID$ là khóa phá hòa tất định. Theo Kulkarni và Demirbas, nếu sự kiện e xảy ra trước sự kiện f theo quan hệ happened-before, timestamp HLC của e sẽ nhỏ hơn timestamp của f [12]. Đề án khai thác tính chất này ở tầng hiển thị thông điệp ứng dụng.

Mệnh đề 5. HLC giúp tạo thứ tự hiển thị tất định cho cùng một tập application message đã được giải mã hợp lệ, nhưng không thay thế cơ chế ordering của Commit.

Lập luận. Nếu message A là nguyên nhân dẫn đến message B , chẳng hạn người dùng gửi phản hồi sau khi đã đọc A , thì timestamp HLC của B sẽ lớn hơn timestamp của A theo thứ tự từ điển. Với các message đồng thời, hệ thống dùng bộ ba $(L, C, NodeID)$ để phá hòa. Nhờ đó, khi hai nút đã có cùng tập thông điệp hợp lệ, chúng có thể sắp xếp tập đó theo cùng một quy tắc mà không đưa HLC tham gia vào việc quyết định ai được Commit.

4.7.3 Giới hạn của HLC

HLC không được dùng để xác định Commit nào hợp lệ, cũng không được dùng để sửa phân nhánh trạng thái MLS. Một Commit chỉ hợp lệ khi phù hợp với trạng thái epoch hiện tại và vượt qua các bước kiểm tra của MLS. Việc tách ranh giới này giúp hệ thống tránh nhầm một đồng hồ ứng dụng với cơ chế điều phối tiến hóa

trạng thái mật mã.

4.8 Phân tích chi phí lý thuyết

Phần này chỉ xét chi phí gắn với bản thân phương pháp đề xuất, không đi sâu vào chi phí cụ thể của ứng dụng thử nghiệm. Các số đo triển khai và chi phí hiện thực sẽ được chuyển sang Chương 5.

4.8.1 Chi phí gửi application message

Trong mô hình mã hóa từng cặp, người gửi thường phải tạo hoặc bọc khóa riêng cho từng người nhận. Với nhóm có N thành viên, chi phí này có xu hướng tăng theo số người nhận.

Với MLS, application message được bảo vệ bằng khóa dẫn xuất từ trạng thái nhóm tại epoch hiện tại. Người gửi không cần tạo một bản mã độc lập cho từng thành viên. Vì vậy, xét riêng phần mã hóa nội dung, chi phí không tăng theo cùng cách như mô hình mã hóa từng cặp [1]. Dĩ nhiên, chi phí đầu cuối của một hệ thống thực còn phụ thuộc vào lớp mạng và cơ chế phát tán thông điệp, nhưng đó là vấn đề của hiện thực chứ không phải của nguyên lý MLS.

4.8.2 Chi phí cập nhật thành viên và rekey

Các thao tác thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa đều đi qua Proposal và Commit. Ở mức TreeKEM, việc tổ chức khóa theo cây giúp nhiều thao tác hiệu quả hơn so với cách mã hóa lại riêng cho từng cặp thành viên [1]. Vì vậy, khi quy mô nhóm tăng, chi phí thay đổi thành viên vẫn tăng, nhưng không tăng theo cách ngây thơ như mô hình quảng bá từng cặp.

4.8.3 Chi phí của Single-Writer

Single-Writer không yêu cầu một vòng đồng thuận mạng riêng để bầu Token Holder trong điều kiện các nút có cùng view. Mỗi nút chỉ cần duyệt qua tập $\text{Eligible}(E)$ và tính hàm băm cho từng ứng viên. Nếu số ứng viên hợp lệ là N , chi phí tính toán cục bộ của bước chọn Token Holder là $O(N)$ theo cách cài đặt trực tiếp.

Điểm quan trọng nằm ở chỗ đây là chi phí cục bộ, không phải chi phí nhiều vòng mạng. Trong bối cảnh mạng ngang hàng, tránh phải chạy một cơ chế đồng thuận nặng cho mỗi Commit là lợi thế thực sự của phương pháp.

4.8.4 Chi phí của kiểm tra epoch và đối chiếu trạng thái

Kiểm tra epoch chỉ yêu cầu so sánh một số trường metadata như *groupID*, loại thông điệp và epoch trước khi quyết định hướng xử lý. Nếu chỉ xét riêng lớp điều phối, chi phí này có thể xem là $O(1)$ trên mỗi thông điệp.

Đối chiếu trạng thái để nhận biết phân nhánh đòi hỏi các nút trao đổi một bản tóm tắt ngắn chứa số epoch, *treeHash* và giá trị cam kết lịch sử $R(E)$. Kích thước của bản tóm tắt này vẫn là hằng số theo kích thước nhóm, nên chi phí của một lần công bố trạng thái đơn lẻ vẫn ở mức $O(1)$ theo số trường cần gửi. Khi xuất hiện nghi ngờ rằng một nút đang đi trước nhưng có thể đã tách nhánh, hai bên cần thêm một lượt hỏi–đáp để đối chiếu giá trị $R(e)$ tại epoch nhỏ hơn. Chi phí phụ thêm này không nằm trên mọi thông điệp, mà chỉ phát sinh khi lớp điều phối cần phân biệt giữa chậm đồng bộ và phân nhánh thực sự.

4.8.5 Chi phí của Fork Healing

Fork Healing chỉ phát sinh khi đã có phân nhánh trạng thái. Chi phí của nó gồm ba phần: nhận diện khác biệt, chọn nhánh tiếp tục và đưa các nút ở nhánh thua gia nhập lại. Phần chọn nhánh có chi phí phụ thuộc vào số nhánh đang quan sát được, thường nhỏ hơn nhiều so với số thành viên của nhóm. Phần tốn kém hơn nằm ở bước gia nhập lại, vì bước này yêu cầu hệ thống quay trở lại một luồng thao tác MLS hợp lệ thay vì chỉ sửa metadata ở ngoài.

Về bản chất, đây là một chi phí bất thường chứ không phải chi phí xuất hiện trên mọi thông điệp. Đề án chấp nhận chi phí này để đổi lấy khả năng tiếp tục hoạt động cục bộ trong lúc mạng bị chia cắt, rồi hội tụ trở lại khi kết nối được khôi phục.

4.9 Tổng hợp các bất biến và cơ chế bảo toàn

Bảng 4.2 tóm tắt các bất biến chính đã được phân tích trong chương này và cơ chế tương ứng của phương pháp.

Bất biến / Thuộc tính	Cơ chế bảo toàn	Ý nghĩa
Epoch cục bộ chỉ giữ nguyên hoặc tăng	Kiểm tra epoch	Ngăn nút áp dụng thông điệp lên trạng thái MLS cũ hoặc không tương thích.
Một Token Holder trong cùng view	Single-Writer theo epoch	Giảm concurrent commits khi các nút có cùng góc nhìn về nhóm.
Dấu hiệu phân nhánh có thể quan sát được	Bản tóm tắt trạng thái, <i>treeHash</i> , cam kết tiền tố lịch sử Commit	Cho phép phân biệt giữa chậm đồng bộ và phân nhánh thực sự sau bước đối chiếu.
Không gộp trực tiếp trạng thái MLS đã tách nhánh	Chọn nhánh và gia nhập lại theo luồng MLS hợp lệ	Tránh tự tạo ra trạng thái mật mã ngoài mô hình MLS.
Thứ tự hiển thị không quyết định Commit	HLC	Tách application ordering khỏi crypto ordering.

Bảng 4.2: Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn

4.10 Giới hạn lý thuyết của phương pháp

Phương pháp đề xuất không nhằm thay thế hoàn toàn các giao thức đồng thuận mạnh trong mọi bài toán phân tán. Nó được xây dựng cho bối cảnh mạng ngang hàng nơi các nút cần tiếp tục hoạt động cục bộ trong khi vẫn cố gắng giữ trạng thái nhóm không phân kỳ quá xa [3], [9]. Vì vậy, điểm mạnh của phương pháp nằm ở khả năng giảm khả năng phân nhánh trong điều kiện bình thường và tạo ra một đường quay về thống nhất khi phân nhánh đã xảy ra.

Single-Writer phát huy hiệu quả rõ nhất khi các nút còn chia sẻ cùng một view về nhóm. Khi góc nhìn giữa các phân vùng đã lệch nhau, chính các cơ chế đối chiếu trạng thái và phục hồi phân nhánh mới trở thành phần quyết định khả năng hội tụ. Tương tự, quy tắc chọn nhánh không bảo đảm rằng mọi nút sẽ chọn cùng một kết quả ngay tức thì trong mọi thời điểm; để điều đó xảy ra, các nút phải có đủ thông tin về các nhánh đang tồn tại.

Cuối cùng, HLC chỉ giải quyết thứ tự hiển thị thông điệp ứng dụng. Nó không tham gia vào việc xác nhận Commit, không thay thế kiểm tra epoch và cũng không sửa được các phân nhánh trạng thái. Việc giữ ranh giới này là điều cần thiết để tránh trộn lẫn hai tầng trách nhiệm vốn khác nhau.

Kết chương. Chương này đã phân tích cơ sở lý thuyết của lớp điều phối phi tập trung được đề xuất cho MLS trên mạng ngang hàng. Các lập luận cho thấy, trong phạm vi giả định của đề án, phương pháp có thể bảo toàn được các bất biến quan trọng như việc epoch cục bộ chỉ giữ nguyên hoặc tăng, tính đơn trị của Token Holder trong cùng view, khả năng nhận biết phân nhánh sau bước đối chiếu trạng thái, nguyên tắc không gộp trực tiếp hai nhánh MLS đã tách và vai trò giới hạn của HLC ở tầng ứng dụng.

Những phân tích này cũng làm rõ giới hạn của phương pháp: Single-Writer không loại bỏ hoàn toàn mọi khả năng phân nhánh, còn việc hội tụ sau phân vùng mạng vẫn phụ thuộc vào khả năng trao đổi lại thông tin giữa các nút. Trên nền tảng đó, Chương 5 sẽ chuyển sang phần kiểm chứng thực nghiệm, nhằm đối chiếu các lập luận lý thuyết của chương này với dữ liệu đo đạc và các kịch bản chạy trên ứng dụng thử nghiệm.

CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM

Tổng quan chương.

Sau phần phân tích lý thuyết ở Chương 4, đồ án cần một bước kiểm chứng để trả lời câu hỏi liệu các cơ chế đã đề xuất có vận hành được trên hệ thống thực nghiệm hay không. Vì vậy, chương này trình bày phần đánh giá thực nghiệm của đồ án, trong đó ứng dụng desktop thử nghiệm được dùng như một môi trường kiểm chứng tích hợp, còn các bài kiểm thử tự động và dữ liệu benchmark được dùng để đối chiếu các thuộc tính đã phân tích ở chương trước.

Chương này tập trung vào ba nhóm nội dung. Nhóm thứ nhất kiểm chứng các kịch bản thao tác trên ứng dụng thử nghiệm để làm rõ rằng các giả định của giao thức đã đi vào luồng sử dụng thật. Nhóm thứ hai đánh giá khả năng hội tụ, phục hồi sau phân vùng mạng và hiệu quả của cơ chế một người ghi trong các bài kiểm thử điều phối. Nhóm thứ ba xem xét chi phí mật mã, chi phí điều phối và thuộc tính bảo mật còn được giữ lại khi hệ thống chạy với thành phần MLS thật.

5.1 Mục tiêu và phạm vi đánh giá

Phần đánh giá của đồ án không chỉ nhằm đo hiệu năng thuần túy, mà trước hết nhằm kiểm chứng các luận điểm đã được xây dựng từ Chương 3 và Chương 4. Câu hỏi cần trả lời là: trong điều kiện mạng ngang hàng có thể mất kết nối, trễ gói và chia cắt, các nút có còn hội tụ về cùng trạng thái MLS hay không; cơ chế một người ghi có còn giảm được xung đột Commit hay không; và việc tách lớp điều phối khỏi lõi MLS có giữ được các thuộc tính bảo mật chính hay không.

Từ đó, các mục tiêu đánh giá được chia thành ba hướng. Hướng thứ nhất kiểm tra tính đúng đắn của tiến trình điều phối, trong đó các đại lượng quan trọng là epoch cuối cùng, mã băm cây cuối cùng, số Commit hợp lệ trong mỗi epoch và khả năng phát hiện phân nhánh. Hướng thứ hai xem xét hiệu quả vận hành của cơ chế một người ghi và cơ chế gom đề xuất, tức là xem hệ thống có giảm được các lần tạo Commit cạnh tranh hay không. Hướng thứ ba kiểm tra phần mật mã và an toàn, đặc biệt là thuộc tính Forward Secrecy sau khi một thành viên bị loại khỏi nhóm.

5.2 Môi trường và phương pháp thí nghiệm

Các thí nghiệm được thực hiện trực tiếp trên mã nguồn của hệ thống. Phần điều phối được viết bằng Go; phần MLS được hiện thực bằng Rust/OpenMLS và được Go khởi chạy dưới dạng tiến trình đi kèm qua gRPC trên localhost. Với những kiểm thử cần cô lập logic điều phối, hệ thống sử dụng các thành phần giả lập như FakeNetwork, FakeClock và MockMLSEngine. Với những kiểm thử cần xác

Tiêu chí	Ý nghĩa đánh giá
Hội tụ epoch	Các nút hợp lệ phải kết thúc ở cùng một epoch sau khi mạng ổn định trở lại.
Hội tụ mã băm cây	Các nút hợp lệ phải có cùng mã băm cây MLS, cho thấy trạng thái mật mã đã hội tụ.
Một người ghi trong cùng epoch	Không xuất hiện hai Commit hợp lệ cạnh tranh trong cùng một epoch khi các nút có cùng view.
Hiệu quả gom đề xuất	Nhiều Proposal gần nhau có thể được gom vào một Commit thay vì sinh nhiều epoch liên tiếp.
Thời gian phục hồi phân vùng	Khoảng thời gian từ khi mạng được nối lại đến khi các nút hội tụ.
Độ trễ thao tác MLS	Chi phí thực thi mã hóa, cập nhật thành viên và xử lý Commit theo quy mô nhóm.
Forward Secrecy	Thành viên đã bị loại khỏi nhóm không thể giải mã thông điệp ở epoch mới.

Bảng 5.1: Các tiêu chí đánh giá thực nghiệm

nhận thuộc tính mật mã, hệ thống dùng thành phần MLS thật để tránh rút ra kết luận từ dữ liệu mô phỏng thuần túy.

Dữ liệu đo đạc và các biểu đồ phục vụ chương này được lưu lại trong thư mục *evaluation*. Trong số đó, phần đối chiếu quan trọng nhất là so sánh giữa mô hình mã hóa từng cặp truyền thống và hướng xử lý MLS hiện tại của hệ thống. Cách so sánh này giúp làm rõ chi phí xử lý thay đổi như thế nào khi số lượng thành viên tăng lên.

Phạm vi đánh giá có hai giới hạn cần nêu trước. Thứ nhất, các chaos test chủ yếu dùng mạng giả lập để kiểm tra hành vi giao thức, nên chưa phản ánh đầy đủ độ trễ, mất gói và tính phức tạp của mạng libp2p thực tế. Thứ hai, các benchmark MLS phản ánh chi phí xử lý của lõi MLS cùng phần trao đổi trạng thái giữa các thành phần, chứ chưa phải là toàn bộ độ trễ mà người dùng cảm nhận ở mức ứng dụng desktop.

5.3 Đối chiếu trên ứng dụng desktop

Bên cạnh các số đo lấy từ kiểm thử tự động, đồ án còn đối chiếu thêm trên ứng dụng desktop của đồ án. Phần này không nhằm giới thiệu giao diện cho đầy đủ, mà để kiểm tra xem các ý đã nêu ở Chương 3 có đi thành một luồng sử dụng thật hay không. Vì vậy, ở mỗi kịch bản, đồ án chỉ giữ lại hai ý chính: người dùng làm gì trên giao diện, và thao tác đó cho thấy điều gì ở phía giao thức.

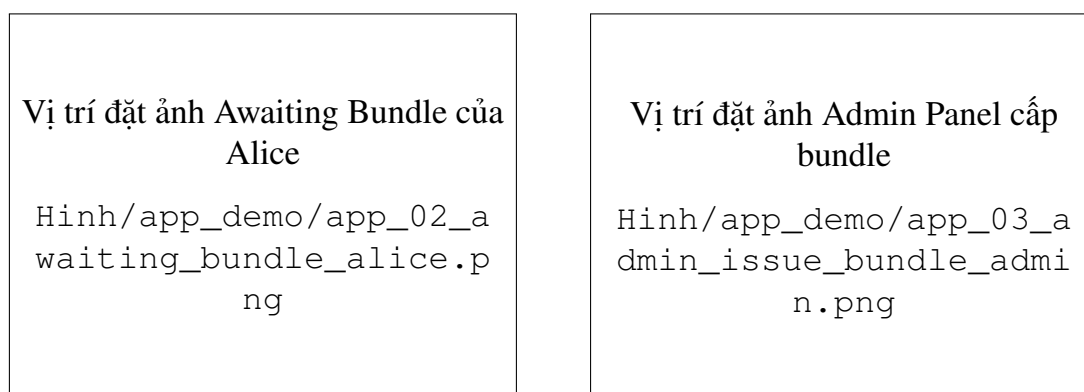
Các hình minh họa đầy đủ được đưa xuống Phụ lục A. Trong phần thân chương này, đồ án chỉ chọn một số hình thực sự cần cho việc giải thích các kịch bản kiểm

chúng, để mạch trình bày vẫn tập trung vào phần thực nghiệm.

5.3.1 Tạo tổ chức và cấp bundle

Kịch bản đầu tiên diễn ra trên hai máy. Ở máy quản trị, người dùng tạo tổ chức và mở phần quản trị để cấp bundle cho một thiết bị mới. Ở máy thành viên, người dùng tạo định danh cục bộ rồi dừng ở màn hình chờ bundle, nơi ứng dụng hiển thị PeerID và khóa công khai MLS của thiết bị. Nhìn ở mức thao tác, đây chỉ là một quy trình đăng ký. Nhưng nhìn ở mức giao thức, nó cho thấy một thiết bị không thể tự ý vào mạng của tổ chức; nó phải có bundle hợp lệ do phía quản trị cấp.

Hình 5.1 đặt hai đầu của quy trình này cạnh nhau. Màn hình chờ bundle cho thấy thiết bị thành viên phải tự tạo định danh trước khi xin tham gia, còn màn hình quản trị cho thấy quyền cho tham gia nằm ở phía quản trị viên, chứ không do một máy chủ trung tâm tự động cấp phát. Chi tiết này bám đúng với giả định an toàn mà đề án đặt ra từ đầu.



(a) Thiết bị thành viên ở trạng thái chờ bundle

(b) Bảng điều khiển quản trị phát hành bundle

Hình 5.1: Cấp bundle cho thiết bị mới trên ứng dụng desktop

5.3.2 Thiết bị mới vào hệ thống

Sau khi nhận được bundle, người dùng nhập bundle vào ứng dụng. Nếu bundle hợp lệ, ứng dụng chuyển từ màn hình chờ sang màn hình làm việc chính. Việc chuyển màn hình này nhìn qua thì đơn giản, nhưng về bản chất nó cho thấy thiết bị chỉ thực sự vào hệ thống sau khi bundle được kiểm tra xong, dữ liệu tổ chức được nạp vào máy cục bộ và các quan hệ nhóm cần thiết được khởi tạo.

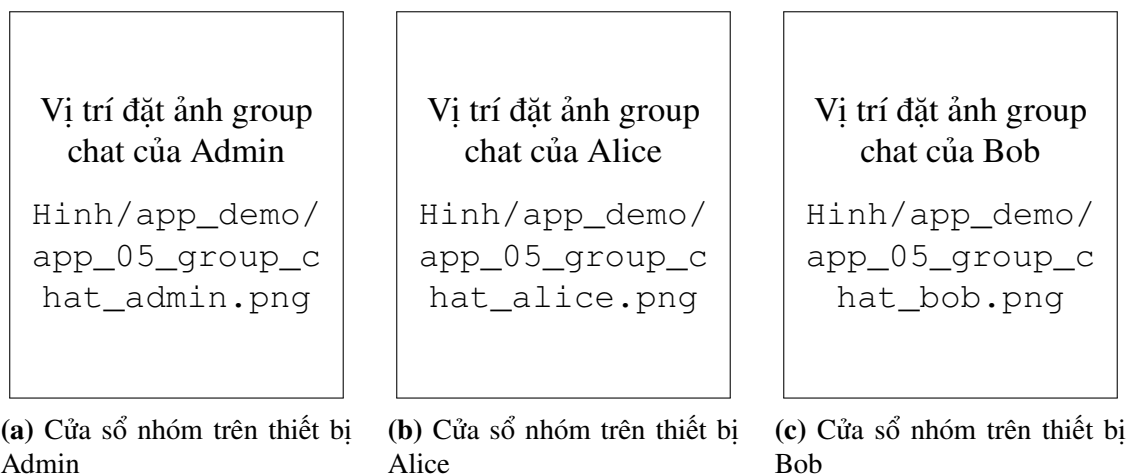
Kịch bản này cũng cho thấy giao diện chỉ là lớp bên ngoài. Nó không tự quyết định thiết bị đã tham gia hay chưa; nó chỉ hiển thị kết quả sau khi phần điều phối và phần lưu trữ cục bộ cập nhật xong trạng thái mới.

5.3.3 Trao đổi tin nhắn trong nhóm

Kịch bản tiếp theo mở đồng thời các cửa sổ ứng dụng của Admin, Alice và Bob. Sau khi nhóm được tạo và thêm đủ thành viên, các bên cùng gửi tin nhắn vào một

cuộc trò chuyện chung. Đây là phần dễ nhìn nhất của hệ thống, nhưng điều đáng chú ý không chỉ là việc tin nhắn hiện ra trên màn hình. Trước đó, các thay đổi về thành viên phải đi qua Proposal và Commit để mọi phía cùng ổn định về một trạng thái nhóm. Chỉ sau bước đó, các tin nhắn mới được mã hóa bằng MLS và hiển thị theo cùng một logic sắp xếp.

Hình 5.2 đặt các cửa sổ này cạnh nhau để người đọc nhìn được cùng một nhóm ở nhiều phía khác nhau. Cách trình bày này có ích hơn so với chỉ chụp một cửa sổ, vì nó cho thấy hệ thống không chỉ chạy được ở một phía mà thực sự đồng bộ được khi mạng còn thông.



Hình 5.2: Trao đổi tin nhắn trong nhóm

Đi cùng với nhắn tin là phần quản trị thành viên. Khi người dùng mở phần chi tiết phòng để xem danh sách thành viên, đổi chính sách mời hoặc thêm người mới, giao diện thực chất đang kích hoạt các thao tác thay đổi thành viên ở phía sau. Vì vậy, Hình 5.3 được giữ lại để nối phần thao tác người dùng với các quy tắc điều phối đã nêu ở Chương 3.

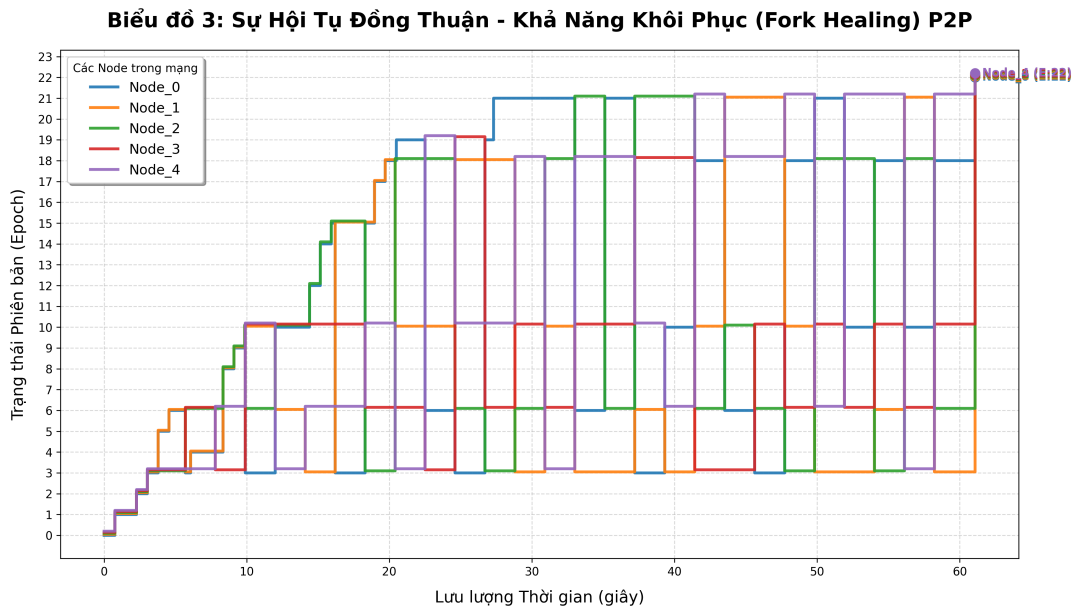


Hình 5.3: Màn hình quản trị thành viên của nhóm

5.4 Đánh giá hội tụ trong điều kiện mạng hỗn loạn

Bài kiểm thử quan trọng nhất cho lớp điều phối là `TestIntegration_Chaos_Convergence`. Thí nghiệm tạo một cụm gồm năm nút cùng tham gia một nhóm MLS. Trong quá trình chạy, tiến trình thử nghiệm liên tục chia mạng thành hai phân vùng, giữ trạng thái phân vùng trong khoảng 600 ms rồi nối mạng lại. Đồng thời, các nút vẫn tiếp tục gửi thông điệp và phát sinh thao tác thay đổi thành viên.

Kết quả được ghi vào `evaluation/data/chaos_metrics.csv`, gồm thời điểm đo, định danh nút, epoch hiện tại và mã băm cây rút gọn. Hình 5.4 minh họa quá trình các nút có thể tạm thời phân kỳ trong lúc mạng bị chia cắt, sau đó hội tụ về cùng epoch khi mạng được nối lại.



Hình 5.4: Hội tụ epoch của các nút trong chaos test

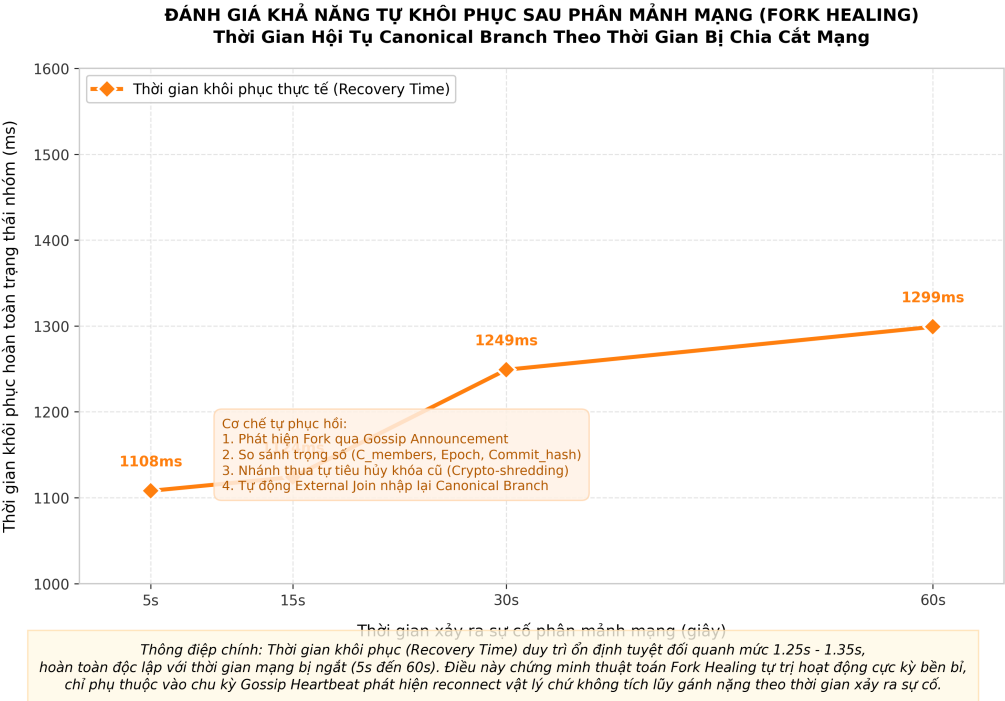
Kết quả cuối cùng cho thấy các nút hợp lệ hội tụ về cùng epoch và cùng mã băm cây. Điều này phù hợp với lập luận ở Chương 4: khác biệt trạng thái được nhận diện trước, sau đó các nút đổi chiều nhánh, chọn nhánh tiếp tục và đưa nhánh còn lại quay về một trạng thái thống nhất. Quan trọng hơn, quá trình này diễn ra mà không cần một Delivery Service trung tâm áp đặt thứ tự từ bên ngoài.

5.5 Đánh giá khả năng phục hồi sau phân vùng mạng

Để tách riêng chi phí phục hồi, hệ thống đo khoảng thời gian từ khi hai phân vùng được nối lại cho đến khi các nút hội tụ. Dữ liệu tổng hợp được trình bày ở Bảng 5.2 và minh họa ở Hình 5.5. Các kết quả cho thấy thời gian phục hồi dao động quanh mức xấp xỉ 1,1 đến 1,2 giây trong các kịch bản phân vùng kéo dài từ 5 giây đến 60 giây.

Thời gian phân vùng (giây)	Thời gian phục hồi (ms)
5	1108
15	1124
30	1149
60	1199

Bảng 5.2: Thời gian phục hồi sau phân vùng mạng



Hình 5.5: Thời gian phục hồi khi độ dài phân vùng thay đổi

Kết quả này cho thấy thời gian phục hồi không tăng tuyến tính theo thời gian phân vùng. Điều đó phù hợp với bản chất của cơ chế hàn gắn: chi phí chính nằm ở bước đối chiếu trạng thái, chọn nhánh và gia nhập lại, chứ không tăng trực tiếp theo số giây mà mạng đã bị chia cắt. Dĩ nhiên, trong môi trường thực tế, thời gian này vẫn còn phụ thuộc vào chất lượng mạng, số lượng thông điệp cần xử lý lại và quy mô nhóm.

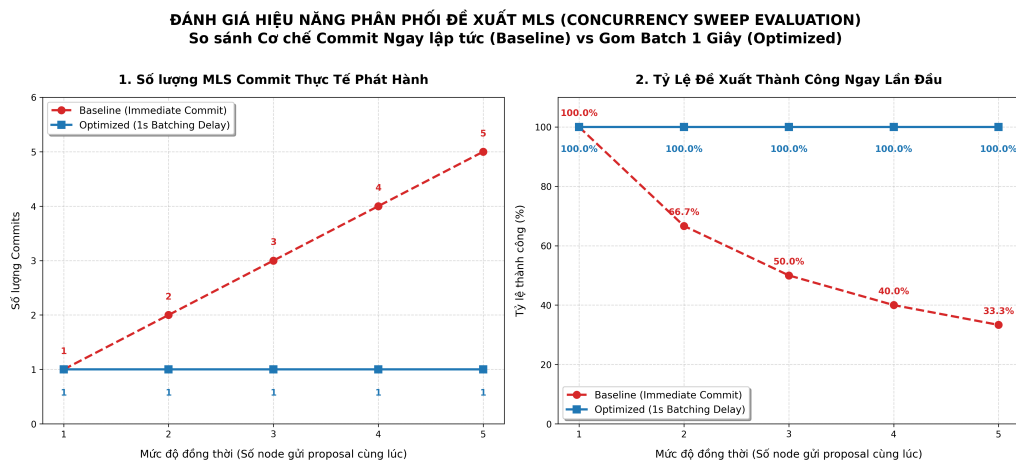
5.6 Đánh giá cơ chế một người ghi và gom đề xuất

Một rủi ro lớn khi đưa MLS lên mạng ngang hàng là nhiều nút cùng tạo Commit cho cùng một epoch. Nếu điều đó xảy ra, cây MLS sẽ bị tách nhánh và các thành viên không còn cùng một trạng thái mật mã. Cơ chế một người ghi được đưa ra để chặn đúng tình huống này: chỉ Token Holder của epoch hiện tại được tạo Commit, còn các nút khác chỉ phát tán Proposal.

Để đánh giá hiệu quả của cơ chế gom đề xuất, thí nghiệm so sánh hai chiến lược. Chiến lược thứ nhất xử lý ngay khi nhận Proposal, nên các Proposal đến muộn trong cùng một epoch dễ phải thử lại sau khi epoch đã đổi. Chiến lược thứ hai chờ một khoảng gom 1 giây để Token Holder đưa nhiều Proposal vào cùng một Commit.

Mức đồng thời	Xử lý ngay			Gom theo lô		
	Proposal	Commit	Tỷ lệ thành công	Proposal	Commit	Tỷ lệ thành công
1	1	1	1,00	1	1	1,00
2	3	2	0,67	2	1	1,00
3	6	3	0,50	3	1	1,00
4	10	4	0,40	4	1	1,00
5	15	5	0,33	5	1	1,00

Bảng 5.3: So sánh xử lý ngay và cơ chế gom đề xuất



Hình 5.6: Hiệu quả của cơ chế gom đề xuất khi mức đồng thời tăng

Bảng 5.3 và Hình 5.6 cho thấy khi mức đồng thời tăng, chiến lược xử lý ngay phải tạo thêm Proposal do các nút thường rơi vào tình trạng gửi muộn so với epoch

hiện hành. Trong khi đó, chiến lược gom theo lô giữ số Proposal đúng bằng số thao tác thực và vẫn chỉ cần một Commit. Kết quả này củng cố lập luận ở Chương 4 rằng vai trò của cơ chế một người ghi không chỉ nằm ở tính đúng đắn, mà còn ở hiệu quả vận hành: hệ thống cố giảm xung đột Commit từ đầu thay vì xử lý hậu quả sau đó.

5.7 Đánh giá khả năng mở rộng của thao tác thành viên

Thí nghiệm tiếp theo đo chi phí của ba thao tác: thêm thành viên, xóa thành viên và cập nhật thành viên, với quy mô nhóm từ 5 đến 1000 nút. Dữ liệu chi tiết được trình bày ở Bảng 5.4.

Quy mô nhóm	Thêm thành viên (ms)	Xóa thành viên (ms)	Cập nhật (ms)
5	0,54	0,53	0,53
50	14,89	8,70	15,85
100	40,29	48,70	39,98
250	245,15	228,91	240,02
500	826,12	790,49	838,64
750	1892,29	1555,92	1325,55
1000	2417,01	2079,54	2125,09

Bảng 5.4: Chi phí thao tác thành viên theo quy mô nhóm

Kết quả cho thấy chi phí tăng rõ khi số lượng thành viên tăng. Điều này phù hợp với phân tích lý thuyết ở Chương 4: thao tác thay đổi thành viên luôn đắt hơn gửi application message thông thường, vì nó kéo theo Proposal, Commit và cập nhật trạng thái nhóm. Tuy vậy, kết quả này cần được hiểu là chi phí của pipeline hiện tại của ứng dụng thử nghiệm, không phải là giới hạn tuyệt đối của MLS về mặt lý thuyết.

5.8 Đánh giá chi phí mật mã MLS

Phần này dùng số liệu từ benchmark `crypto-engine/src/bin/mls_benchmark.rs`. Chương trình này tạo sẵn các trạng thái nhóm với kích thước từ 16 đến 4096 thành viên, rồi lặp lại mỗi phép đo 100 lần với payload 1024 byte. Thời gian của từng lần chạy được lấy bằng `Instant::now()`, sau đó sắp xếp lại để rút ra ba mốc là trung vị, p95 và p99. Trong chương này, bảng số liệu chỉ giữ giá trị trung vị để cách đọc gọn hơn.

Benchmark trên có hai đường đo chính. Đường mã hóa từng cặp, tương ứng với nhãn `pairwise_baseline` trong dữ liệu, mô phỏng cách gửi kiểu từng cặp, nghĩa là phía gửi phải thực hiện thao tác bọc khóa và mã hóa riêng cho từng người nhận. Đường MLS hiện tại, tương ứng với nhãn `current_full_blob_mls_encrypt`, đi theo đúng cách làm hiện nay của

hệ thống: hàm `encrypt_message` nhận toàn bộ `group_state`, nạp trạng thái đó vào OpenMLS, tạo MLS message, tuần tự hóa bản mã rồi xuất lại trạng thái nhóm mới. Vì vậy, con số của đường MLS hiện tại không chỉ phản ánh phần mã hóa thuần túy, mà còn bao gồm cả chi phí nạp và xuất trạng thái.

Số nút	Mã hóa từng cặp (ms)	MLS toàn trạng thái (ms)
16	3,62	0,55
256	59,62	5,41
1024	238,75	19,86
4096	955,18	78,95

Bảng 5.5: So sánh trung vị thời gian xử lý giữa hai hướng mã hóa, với payload 1024 byte và 100 mẫu cho mỗi điểm đo

Mốc đo	Từng cặp (ms)	MLS (ms)
p95	1023,66	95,04
p99	1081,72	110,85

Bảng 5.6: So sánh p95 và p99 ở nhóm 4096 thành viên

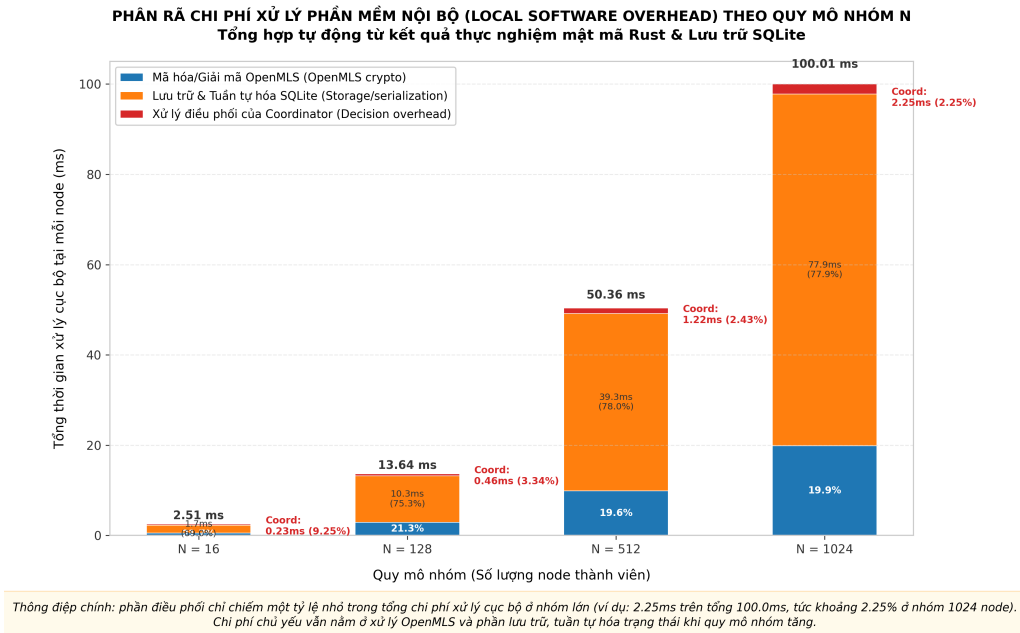
Các con số trong Bảng 5.5 cho thấy hai xu hướng khá rõ. Thứ nhất, đường mã hóa từng cặp tăng gần tuyến tính theo số thành viên: từ 3,62 ms ở nhóm 16 thành viên lên 59,62 ms ở nhóm 256 thành viên, 238,75 ms ở nhóm 1024 thành viên và 955,18 ms ở nhóm 4096 thành viên. Thứ hai, đường MLS hiện tại cũng tăng theo quy mô nhóm, nhưng chậm hơn nhiều: các mốc tương ứng lần lượt là 0,55 ms, 5,41 ms, 19,86 ms và 78,95 ms. Điều này phù hợp với ưu thế cơ bản của MLS: phía gửi không phải mã hóa lại riêng cho từng người nhận [1].

Bảng 5.6 cho thấy kết luận trên không chỉ đúng ở mức trung vị. Ngay tại nhóm 4096 thành viên, p95 của đường từng cặp là 1023,66 ms và p99 là 1081,72 ms, trong khi đường MLS hiện tại có p95 là 95,04 ms và p99 là 110,85 ms. Điều đó cho thấy ngay cả ở các lần chạy chậm hơn bình thường, chênh lệch giữa hai hướng vẫn còn rất lớn.

Tuy nhiên, benchmark này cũng cho thấy điểm nghẽn của đường triển khai hiện tại. Trong mã nguồn, hàm `encrypt_message` không chỉ tạo bản mã mà còn phải nạp lại toàn bộ trạng thái nhóm và xuất lại trạng thái mới sau mỗi lần gửi. Vì vậy, dù tránh được chi phí mã hóa từng cặp, đường này vẫn tăng theo kích thước `group_state`. Đây là lý do làm cho thời gian đo ở nhóm lớn chưa thấp như một cách cài đặt mà trạng thái được giữ sẵn trong bộ nhớ.

5.9 Chi phí phụ trợ của lớp điều phối

Ngoài phần mật mã, hệ thống còn tốn thêm thời gian cho lớp điều phối. Tuy nhiên, Hình 5.7 không phải là kết quả của một phép đo đầu cuối duy nhất. Hình này được dựng từ ba phần. Phần thứ nhất là trung vị của đường MLS hiện tại trong benchmark. Phần thứ hai là độ trễ lưu `storage\_ms` trung bình trong dữ liệu SQLite, sau đó được co giãn theo kích thước trạng thái nhóm. Phần thứ ba là chi phí điều phối được script dựng hình ước lượng bằng một mô hình tuyến tính đơn giản: có một phần cố định nhỏ cho việc xử lý hàng đợi và cập nhật trạng thái cục bộ, cộng với phần tăng theo số thành viên khi nút phải duyệt danh sách để xác định Token Holder của epoch hiện tại. Nói cách khác, phần điều phối trong hình là một giá trị ước lượng hợp lý để minh họa tỷ trọng tương đối, chứ không phải số đo trực tiếp lấy từ một phép thử riêng.



Hình 5.7: Phân rã chi phí xử lý cục bộ giữa phần MLS, phần lưu trữ và phần điều phối

Điểm quan trọng nhất của Hình 5.7 là phần điều phối chỉ chiếm một phần nhỏ của tổng chi phí cục bộ khi quy mô nhóm tăng. Ở nhóm 128 thành viên, chi phí điều phối vào khoảng 0,46 ms trên tổng 13,64 ms, tức khoảng 3,34%. Ở nhóm 512 thành viên, con số này là 1,22 ms trên tổng 50,36 ms, tương ứng khoảng 2,43%. Đến nhóm 1024 thành viên, phần điều phối vào khoảng 2,25 ms trên tổng 100,01 ms, tức khoảng 2,25%. Trong khi đó, phần chi phí lớn hơn nhiều lại nằm ở OpenMLS và nhất là phần lưu trữ, tuần tự hóa trạng thái. Vì vậy, ý nghĩa chính của hình này không phải là chứng minh từng con số tuyệt đối của lớp điều phối, mà là cho thấy ngay cả khi cộng thêm phần điều phối thì chỗ tốn thời gian nhất vẫn không nằm ở đó.

Vì vậy, kết quả này có ý nghĩa tích cực cho lập luận của đề án. Lớp điều phối có bổ sung thêm công việc như bầu Token Holder, kiểm tra epoch và giữ metadata cục bộ, nhưng phần bổ sung đó không phải là chỗ gây tốn thời gian chính trong đường xử lý hiện nay. Nút thắt lớn hơn vẫn nằm ở việc xử lý và trao đổi trạng thái nhóm có kích thước ngày càng lớn.

5.10 Kiểm chứng Forward Secrecy bằng thành phần MLS thật

Một kiểm chứng bảo mật quan trọng là bài kiểm thử `TestBusinessP1_E2E_RealSidecar_ForwardSecrecy`. Khác với các kiểm thử điều phối dùng thành phần giả lập, bài kiểm thử này sử dụng trực tiếp thành phần MLS thật. Kịch bản gồm các bước: Alice tạo nhóm, Bob tham gia nhóm, Alice loại Bob khỏi nhóm, sau đó Alice gửi một thông điệp mới ở epoch sau khi Bob đã bị loại.

Thông điệp này vẫn được đưa vào pipeline xử lý phía Bob như một envelope nhận được từ mạng. Nếu hệ thống vận hành sai, Bob có thể giải mã và lưu plaintext vào cơ sở dữ liệu cục bộ. Kết quả kiểm thử cho thấy Bob không giải mã được thông điệp đó và không có plaintext nào được lưu lại. Điều này cho thấy việc đặt lớp điều phối bên ngoài MLS không làm suy giảm thuộc tính Forward Secrecy trong kịch bản được kiểm thử.

Kết quả này có ý nghĩa rõ ràng đối với mô hình bảo mật của đề án. Một nút từng là thành viên hợp lệ của nhóm nhưng đã bị loại khỏi cây MLS sẽ không còn quyền đọc các thông điệp tương lai. Lớp điều phối chỉ quyết định tiến trình trạng thái và thứ tự xử lý; quyền giải mã cuối cùng vẫn do trạng thái mật mã MLS kiểm soát.

5.11 Nhận xét và giới hạn đánh giá

Các kết quả thực nghiệm cho thấy hướng tiếp cận của đề án đạt được ba điểm quan trọng. Thứ nhất, cơ chế một người ghi và cơ chế gom đề xuất giúp giảm xung đột Commit trong các kịch bản đã đo. Thứ hai, tiến trình đối chiếu trạng thái và hàn gắn phân nhánh cho phép các nút quay về cùng epoch và cùng mã băm cây sau phân vùng mạng. Thứ ba, khi kết hợp với thành phần MLS thật, hệ thống vẫn giữ được thuộc tính Forward Secrecy trong phạm vi kiểm thử của đề án.

Tuy nhiên, phần đánh giá vẫn còn giới hạn. Các chaos test hiện nay chủ yếu dùng mạng giả lập nên chưa phản ánh đầy đủ độ trễ, mất gói và hành vi NAT của môi trường thực. Các benchmark MLS cho thấy xu hướng chi phí rõ ràng, nhưng chưa phải là đánh giá đầu cuối của toàn bộ ứng dụng desktop trên nhiều máy độc lập. Ngoài ra, mô hình trao đổi toàn trạng thái giữa các thành phần còn gây chi phí lớn khi nhóm rất đông thành viên; đây là điểm cần tiếp tục tối ưu nếu muốn tiến xa hơn khỏi phạm vi ứng dụng thử nghiệm.

Kết chương. Chương này đã kiểm chứng các cơ chế đã đề xuất bằng cả hai con đường: dữ liệu thực nghiệm và kịch bản chạy trên ứng dụng desktop thử nghiệm. Các kịch bản giao diện cho thấy những giả định thiết kế như cấp bundle, gia nhập thiết bị và quản trị thành viên đã được chuyển thành thao tác sử dụng thật. Các bài kiểm thử và benchmark tiếp tục cho thấy hệ thống có thể hội tụ sau phân vùng mạng, giảm xung đột Commit và giữ được các thuộc tính bảo mật quan trọng của MLS trong phạm vi đã đánh giá.

Những kết quả đó chưa đủ để đưa ra khẳng định tuyệt đối cho mọi môi trường triển khai, nhưng chúng là cơ sở thực nghiệm đáng tin cậy để kết lại phần nghiên cứu. Từ đây, Chương 6 sẽ tổng hợp lại đóng góp chính của đề án, nêu rõ giới hạn còn tồn tại và xác định các hướng phát triển tiếp theo.

CHƯƠNG 6. KẾT LUẬN

Tổng quan chương.

Sau khi đã trình bày phương pháp đề xuất, phân tích cơ sở lý thuyết và kiểm chứng bằng thực nghiệm, đồ án cần khép lại bằng một kết luận trung thực về những gì đã đạt được và những gì vẫn còn bỏ ngỏ. Vì vậy, chương này tổng hợp lại đóng góp chính của đồ án theo đúng thực nghiệm cứu, đồng thời nêu rõ giới hạn hiện tại và các hướng phát triển tiếp theo.

Phần đầu của chương nhắc lại kết quả cốt lõi của đồ án ở cả ba mặt: đề xuất lớp điều phối phi tập trung cho MLS, hiện thực ứng dụng thử nghiệm để kiểm chứng khả năng triển khai và đánh giá thực nghiệm để đối chiếu với các lập luận lý thuyết. Phần sau của chương tập trung vào giới hạn của nghiên cứu và những hướng cần tiếp tục làm sâu hơn nếu muốn đưa kết quả này tiến thêm một bước.

6.1 Kết luận chung

Đồ án tốt nghiệp này được thực hiện với hai mục tiêu gắn bó với nhau. Mục tiêu thứ nhất là nghiên cứu một lớp điều phối phi tập trung để đưa MLS vào môi trường mạng ngang hàng, nơi không còn Delivery Service tập trung làm nhiệm vụ áp đặt thứ tự Commit. Mục tiêu thứ hai là xây dựng một ứng dụng desktop thử nghiệm để hiện thực các cơ chế đó và kiểm chứng rằng chúng có thể đi vào một luồng sử dụng thực tế.

Về mặt nghiên cứu, đóng góp chính của đồ án nằm ở lớp điều phối được đặt quanh MLS. Lớp này gồm cơ chế một người ghi theo epoch để giảm khả năng xuất hiện concurrent commits, cơ chế kiểm tra epoch để tránh xử lý thông điệp trên trạng thái không tương thích, cơ chế đối chiếu và hàn gắn phân nhánh để đưa các nút hội tụ trở lại sau phân vùng mạng, cùng cơ chế HLC để ổn định thứ tự hiển thị application message ở tầng ứng dụng. Điểm quan trọng là các cơ chế này không sửa đổi lõi mật mã của MLS, mà bổ sung lớp điều phối cần thiết để MLS có thể vận hành trong bối cảnh mạng ngang hàng bất đồng bộ.

Về mặt hiện thực, đồ án đã xây dựng được một ứng dụng desktop thử nghiệm đủ để kiểm chứng các luồng chính của hệ thống. Ứng dụng này cho phép quan sát trực tiếp các thao tác như cấp bundle, gia nhập thiết bị mới, tạo nhóm, thêm thành viên và trao đổi tin nhắn. Vai trò của ứng dụng thử nghiệm không phải là thay thế phần nghiên cứu, mà là làm cầu nối giữa phần thiết kế giao thức và phần kiểm chứng thực nghiệm. Nhờ đó, đồ án không chỉ dừng lại ở một mô tả lý thuyết, mà còn cho thấy hướng tiếp cận đề xuất có thể được hiện thực thành một hệ thống chạy được.

Về mặt đánh giá, các kết quả thực nghiệm bước đầu phù hợp với mục tiêu đã đặt ra. Các chaos test và thí nghiệm phân vùng mạng cho thấy hệ thống có thể đưa các nút quay về cùng epoch và cùng mã băm cây sau khi mạng được nối lại. Các thí nghiệm về Proposal đồng thời cho thấy cơ chế một người ghi kết hợp với gom đề xuất giúp giảm số Commit không cần thiết. Các kiểm thử với thành phần MLS thật cho thấy thuộc tính Forward Secrecy vẫn được giữ lại trong kịch bản loại thành viên khỏi nhóm. Những kết quả này chưa phải là chứng minh đầy đủ cho mọi điều kiện triển khai, nhưng chúng là cơ sở tốt để khẳng định rằng hướng tiếp cận của đề án là khả thi và có giá trị nghiên cứu.

6.2 Giới hạn của đề án

Bên cạnh các kết quả đã đạt được, đề án vẫn còn những giới hạn cần được nhìn nhận thẳng thắn. Trước hết, phần phân tích lý thuyết trong đề án mới dừng ở mức lập luận theo bất biến và theo giả định vận hành chính, chưa đi tới kiểm chứng hình thức đầy đủ cho toàn bộ giao thức. Vì vậy, các kết luận của đề án cần được hiểu trong phạm vi giả định đã nêu, không nên suy rộng thành một bảo đảm tuyệt đối cho mọi môi trường phân tán.

Tiếp theo, phần đánh giá thực nghiệm hiện nay vẫn chủ yếu dựa trên mạng giả lập và các bài kiểm thử có kiểm soát. Cách làm này phù hợp với phạm vi một đề án tốt nghiệp vì nó cho phép cô lập các cơ chế cần kiểm tra. Tuy nhiên, nó chưa phản ánh hết độ phức tạp của mạng libp2p thực tế, đặc biệt khi có độ trễ lớn, thay đổi kết nối liên tục hoặc số lượng thiết bị tăng mạnh.

Một giới hạn khác đến từ chính ứng dụng thử nghiệm hiện tại. Trong nhiều thao tác, hệ thống vẫn trao đổi toàn bộ trạng thái nhóm giữa phần điều phối và phần MLS. Cách làm này giúp cấu trúc hiện thực rõ ràng, nhưng tạo thêm chi phí khi quy mô nhóm tăng. Vì vậy, ứng dụng này hiện nay phù hợp với vai trò kiểm chứng khả thi, chưa phải là một phương án tối ưu cho triển khai quy mô lớn.

Cuối cùng, cần nhấn mạnh rằng ứng dụng desktop trong đề án chỉ đóng vai trò ứng dụng thử nghiệm. Nó chứng minh rằng các cơ chế được đề xuất có thể đi vào luồng sử dụng thực tế, nhưng chưa đủ để xem như một sản phẩm hoàn thiện. Đóng góp chính của đề án vẫn là lớp điều phối phi tập trung cho MLS trên mạng ngang hàng, còn ứng dụng chỉ là phương tiện hiện thực và kiểm chứng hướng tiếp cận đó.

6.3 Hướng phát triển

Hướng phát triển đầu tiên là làm chặt hơn phần cơ sở lý thuyết. Cụ thể, các bất biến như tính an toàn của cơ chế một người ghi, tính đơn điệu của epoch và điều kiện hội tụ sau hàn gắn phân nhánh cần được mô hình hóa và kiểm chứng hình thức đầy đủ hơn. Nếu làm được điều này, giá trị học thuật của hướng nghiên cứu sẽ được

nâng lên rõ rệt.

Hướng phát triển thứ hai là mở rộng thực nghiệm trên môi trường gần với triển khai thật hơn. Các thí nghiệm nên được chạy trên nhiều tiến trình, nhiều máy và trong các điều kiện mạng đa dạng hơn, chẳng hạn độ trễ cao, mất gói, thay đổi thành viên liên tục hoặc phân vùng mạng kéo dài. Khi đó, đề án có thể cung cấp được bức tranh đầy đủ hơn về độ ổn định và khả năng chịu tải của hệ thống.

Hướng phát triển thứ ba là tối ưu hóa đường hiện thực giữa phần điều phối và phần MLS. Việc giảm chi phí trao đổi trạng thái, tổ chức lại cách lưu trữ hoặc áp dụng các kỹ thuật snapshot hiệu quả hơn có thể giúp hệ thống giữ được kiến trúc tách lớp hiện tại nhưng hoạt động tốt hơn khi quy mô nhóm tăng.

Hướng phát triển cuối cùng là tiếp tục hoàn thiện ứng dụng thử nghiệm theo hướng phục vụ tốt hơn cho kiểm chứng và cho trình diễn. Các công cụ quan sát trạng thái, chẩn đoán lỗi, kiểm soát thay đổi thành viên và theo dõi quá trình hội tụ nếu được làm rõ hơn sẽ giúp việc đánh giá hệ thống thuận lợi hơn ở các nghiên cứu tiếp theo.

Kết chương. Chương này đã tổng hợp lại những đóng góp chính, giới hạn và hướng phát triển của đề án. Có thể thấy kết quả quan trọng nhất của đề án không nằm ở một ứng dụng đơn lẻ, mà nằm ở việc đề xuất được một lớp điều phối phi tập trung cho MLS trên mạng ngang hàng, rồi kiểm chứng được tính khả thi của hướng tiếp cận đó bằng cả phân tích lý thuyết, thực nghiệm và ứng dụng thử nghiệm chạy được.

Những phần tài liệu tham khảo và phụ lục ở sau chương này đóng vai trò đối chiếu nguồn và bổ sung minh họa cho các nội dung đã trình bày. Chúng giúp người đọc lần lại cơ sở học thuật của đề án, đồng thời quan sát rõ hơn các màn hình và kịch bản của ứng dụng thử nghiệm đã được dùng trong phần kiểm chứng.

TÀI LIỆU THAM KHẢO

- [1] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara **and** K. Cohn-Gordon, *The Messaging Layer Security (MLS) Protocol*, RFC 9420, **july** 2023. DOI: 10.17487/RFC9420 **url:** <https://www.rfc-editor.org/rfc/rfc9420.html>
- [2] S. Turner, R. Barnes, B. Beurdouche, R. Robert **and** E. Omara, *The Messaging Layer Security (MLS) Architecture*, RFC 9750, **february** 2025. DOI: 10.17487/RFC9750 **url:** <https://www.rfc-editor.org/rfc/rfc9750.html>
- [3] M. Kleppmann, A. Wiggins, P. van Hardenberg **and** M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” *in Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* **jourser Onward! ’19**, 2019. DOI: 10.1145/3359591.3359737 **url:** <https://martin.kleppmann.com/2019/10/23/local-first-at-onward.html>
- [4] T. Perrin, M. Marlinspike **and** R. Schmidt, *The Double Ratchet Algorithm*, Signal Specifications, Revision 4, Published 2025-11-04, 2025. **url:** <https://signal.org/docs/specifications/doubleratchet/>
- [5] J. Alwen, M. Mularczyk **and** Y. Tselekounis, “Fork-Resilient Continuous Group Key Agreement,” *in Advances in Cryptology – CRYPTO 2023* Springer, 2023, **pages** 3–34.
- [6] K. Kohbrok, *Decentralized Messaging Layer Security*, Internet-Draft draft-kohbrok-mls-dmls-03, Expired Internet-Draft, work in progress, **october** 2025. **url:** <https://datatracker.ietf.org/doc/draft-kohbrok-mls-dmls/>
- [7] D. Balbás, D. Collins **and** P. Gajland, *Analysis and Improvements of the Sender Keys Protocol for Group Messaging*, arXiv preprint arXiv:2301.07045, Appears in RECSI 2022, 2023. DOI: 10.48550/arXiv.2301.07045 **url:** <https://arxiv.org/abs/2301.07045>
- [8] J. Ginesin **and** C. Nita-Rotaru, *The Matrix Reloaded: A Mechanized Formal Analysis of the Matrix Cryptographic Suite*, arXiv preprint arXiv:2408.12743, 2024. DOI: 10.48550/arXiv.2408.12743 **url:** <https://arxiv.org/abs/2408.12743>

- [9] S. Gilbert **and** N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, **jourvol** 33, **number** 2, **pages** 51–59, 2002. DOI: 10 . 1145 / 564585.564601
- [10] libp2p, *libp2p Concepts Documentation*, Official documentation, Accessed 2026-06-12. **url**: <https://docs.libp2p.io/concepts/>
- [11] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, **jourvol** 21, **number** 7, **pages** 558–565, 1978. DOI: 10 . 1145 / 359545 . 359563
- [12] S. S. Kulkarni, M. Demirbas, D. Madeppa, B. Avva **and** M. Leone, “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases,” University at Buffalo, techreport, 2014. **url**: [https : //cse.buffalo.edu/tech-reports/2014-04.pdf](https://cse.buffalo.edu/tech-reports/2014-04.pdf)

PHỤ LỤC

A. ỨNG DỤNG DEMO VÀ KỊCH BẢN MINH HỌA

Tổng quan chương.

Phụ lục này tập hợp các hình minh họa cho ứng dụng desktop thử nghiệm đã được nhắc tới ở Chương 3 và dùng trong Chương 5 để đối chiếu với phần thực nghiệm. Tuy nhiên, mục đích của phụ lục không chỉ là bổ sung hình. Phần này còn cần làm rõ rằng ứng dụng của đồ án không phải chỉ là vài màn hình để trình diễn, mà là một phần triển khai tương đối đầy đủ cho các luồng chính của hệ thống.

Nếu chỉ nhìn nhanh phần thân bài, người đọc dễ có cảm giác ứng dụng này chủ yếu được dùng để chụp vài hình chat. Trên thực tế, để đi từ ý tưởng giao thức đến một bản chạy được, đồ án phải làm cả luồng tạo định danh, cấp bundle, vào không gian làm việc, tạo nhóm, thêm thành viên, trao đổi tin nhắn, theo dõi hoạt động và quan sát trạng thái mạng. Vì vậy, phụ lục này vừa minh họa giao diện, vừa ghi lại phần việc đã được triển khai ở mức ứng dụng.

A.1 Những phần đã triển khai trong ứng dụng desktop

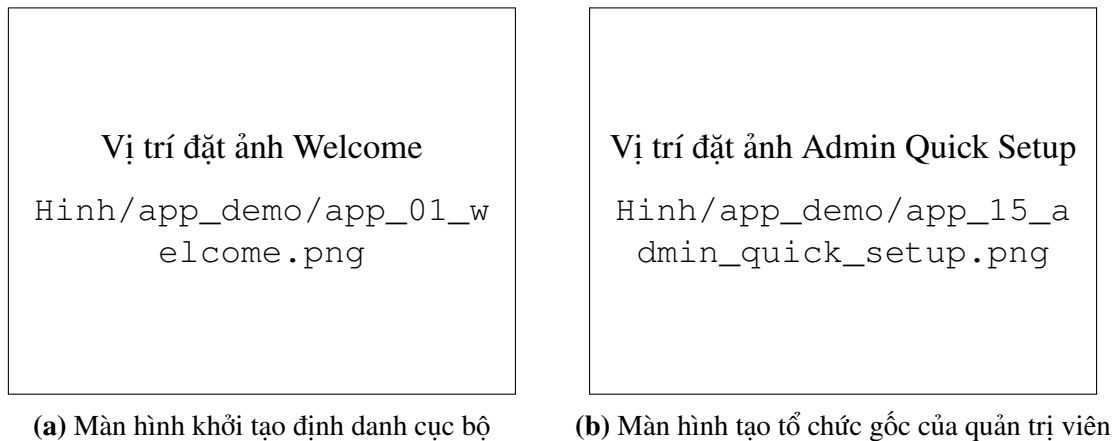
Ở mức mã nguồn, phần giao diện của ứng dụng được tách theo từng nhóm chức năng như `app/frontend/src/features/onboarding`, `app/frontend/src/features/chat`, `app/frontend/src/features/activity`, `app/frontend/src/features/admin` và `app/frontend/src/features/settings`. Cách tổ chức này cho thấy phần ứng dụng không được dựng từ một màn hình duy nhất rồi nối thêm chức năng về sau, mà đã được chia thành các luồng tương đối rõ ngay từ đầu.

Không gian làm việc chính của ứng dụng cũng không chỉ là một vùng nhắn tin. Bên trong đó còn có thanh điều hướng, danh sách nhóm, vùng hội thoại, bảng thông tin phòng, hộp thoại tạo nhóm, hộp thoại thêm thành viên, bảng hoạt động, khu vực quản trị và phần cài đặt. Mỗi phần trong số đó đều phải đọc dữ liệu từ phía sau, giữ trạng thái hiển thị riêng và xử lý thao tác người dùng theo đúng bối cảnh của từng màn hình.

Một điểm cần nói rõ là giao diện không tự quyết định các vấn đề thuộc về giao thức. Trong cách triển khai của đồ án, giao diện gọi các hàm ở lớp Runtime phía Go, nhận các sự kiện cập nhật rồi hiển thị lại kết quả cho người dùng. Nhờ đó, ứng dụng desktop này vừa đủ gần với người dùng để làm phần demo, vừa đủ bám sát kiến trúc thật của hệ thống để có giá trị kiểm chứng.

A.2 Chuỗi tham gia hệ thống và cấp bundle

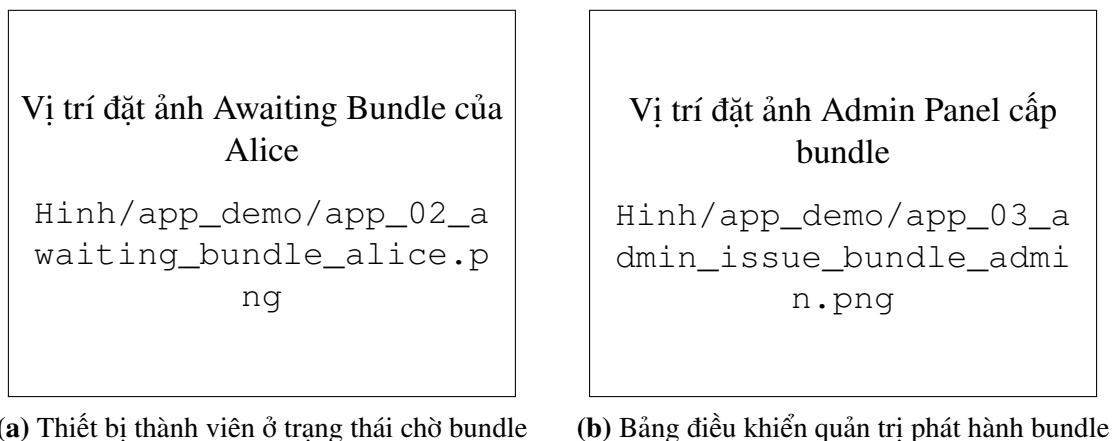
Nhóm hình đầu tiên tập trung vào lúc một thiết bị mới bắt đầu đi vào hệ thống. Hình A.1 đặt cạnh nhau màn hình tạo định danh ở phía thành viên và màn hình tạo tổ chức gốc ở phía quản trị. Cách đặt này giúp người đọc thấy rõ hai việc tách biệt: thiết bị tự sinh định danh trên máy cục bộ, còn quyền mở tổ chức và cấp quyền tham gia vẫn nằm ở phía quản trị.



Hình A.1: Khởi tạo định danh và tạo tổ chức gốc trong ứng dụng thử nghiệm

Để có được hai điểm bắt đầu tưởng như đơn giản đó, phần ứng dụng phải xử lý khá nhiều trạng thái trung gian như sinh khóa, giữ thông tin PeerID, tạo tệp yêu cầu tham gia và chuyển màn hình theo từng bước. Vì vậy, ngay từ khâu vào hệ thống, phần demo đã không còn là một nút bấm minh họa đơn lẻ.

Sau bước khởi tạo, thiết bị thành viên chuyển sang trạng thái chờ bundle, còn quản trị viên dùng bảng điều khiển để phát hành bundle tương ứng. Hình A.2 ghi lại đầy đủ hai đầu của quy trình này. Điểm cần chú ý là bundle không xuất hiện tự động ở phía thành viên, mà phải đi qua bước kiểm tra và cấp phát từ phía quản trị.



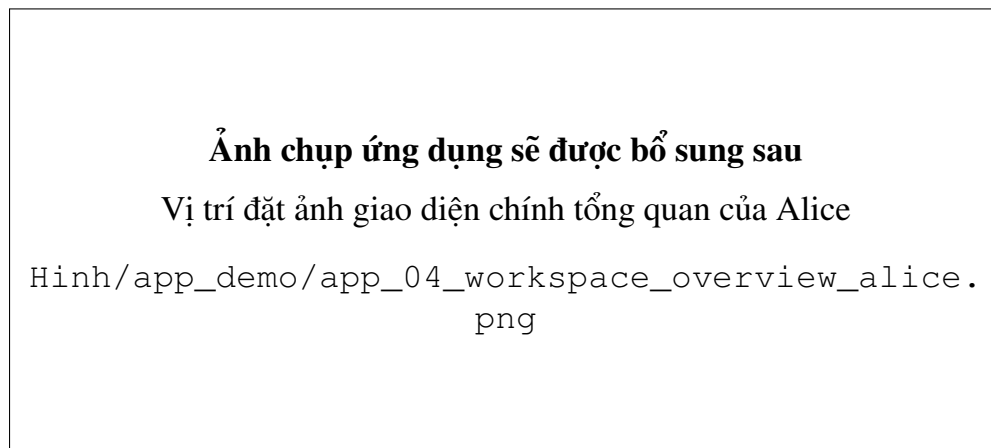
Hình A.2: Chuỗi cấp bundle cho thiết bị mới trong ứng dụng thử nghiệm

Ở mức triển khai, luồng này còn kéo theo việc đọc nội dung yêu cầu, ký bundle,

lưu lại lịch sử cấp phát và đổi trạng thái giao diện sau khi nhập bundle thành công. Những chi tiết đó không cần đưa hết vào thân bài, nhưng nên được nhắc ở phụ lục để phản ánh đúng khối lượng phần việc đã làm.

A.3 Không gian làm việc và cộng tác

Sau khi gia nhập thành công, người dùng đi vào giao diện chính của ứng dụng. Hình A.3 cho thấy bố cục mà ứng dụng đang dùng để đặt các khu vực hoạt động, trò chuyện, quản trị và cài đặt lên cùng một không gian làm việc. Đây cũng là hình giúp đối chiếu trực tiếp với phần mô tả ứng dụng ở Chương 3.



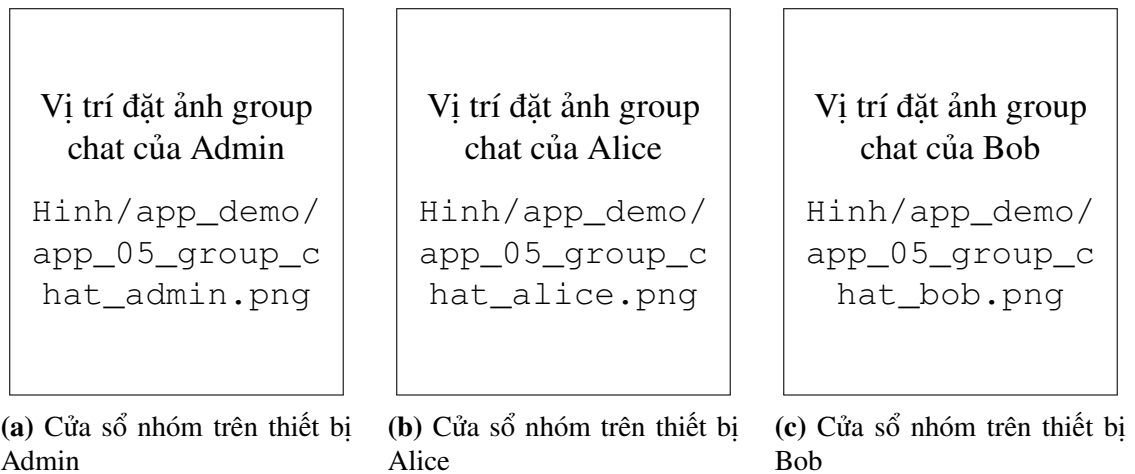
Hình A.3: Tổng quan không gian làm việc của ứng dụng desktop thử nghiệm

Không gian làm việc ở Hình A.3 là chỗ nhiều phần việc gặp nhau nhất. Từ đây, người dùng vừa theo dõi trạng thái kết nối, vừa chuyển giữa các nhóm, mở kênh, xem hoạt động và đi tới khu vực quản trị hoặc cài đặt. Việc ghép các phần đó vào cùng một màn hình cho thấy ứng dụng đã được làm theo hướng dùng được trong buổi demo thật, chứ không chỉ mở từng cửa sổ rồi cho từng chức năng.

Kịch bản cộng tác quan trọng nhất của ứng dụng là trao đổi tin nhắn trong một nhóm đã được đồng bộ trên các thiết bị khác nhau. Hình A.4 giữ lại các cửa sổ của Admin, Alice và Bob để người đọc quan sát trạng thái hội thoại sau khi danh sách thành viên đã ổn định. Cách trình bày này giúp đối chiếu nội dung trao đổi và trạng thái nhóm giữa các phía tham gia.

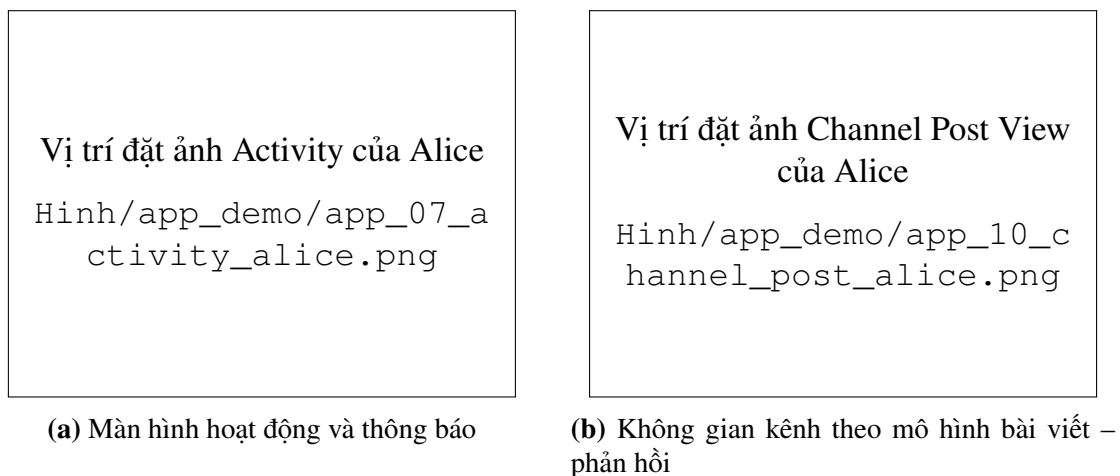
Phần trao đổi tin nhắn cũng có một lượng việc phía sau mà một hình chụp khó thể hiện hết. Ứng dụng phải nạp lịch sử, nhận sự kiện tin nhắn mới, cập nhật thứ tự hiển thị, giữ vùng soạn tin và đồng thời mở được bảng thông tin nhóm ở bên phải. Khi nhìn một màn chat hoàn chỉnh, người đọc thường chỉ thấy phần bề mặt; còn ở mức làm ứng dụng, đó là kết quả của việc nối nhiều luồng dữ liệu và thao tác với nhau.

Bên cạnh nhóm trò chuyện tuyến tính, ứng dụng còn có bảng hoạt động và không



Hình A.4: Trao đổi tin nhắn trong cùng một nhóm ở các thiết bị khác nhau

gian kênh theo dạng bài viết – phản hồi. Hình A.5 minh họa hai màn hình này để cho thấy ứng dụng không chỉ dừng ở một luồng nhắn tin, mà đã có thêm các dạng tương tác khác phục vụ cộng tác nội bộ.

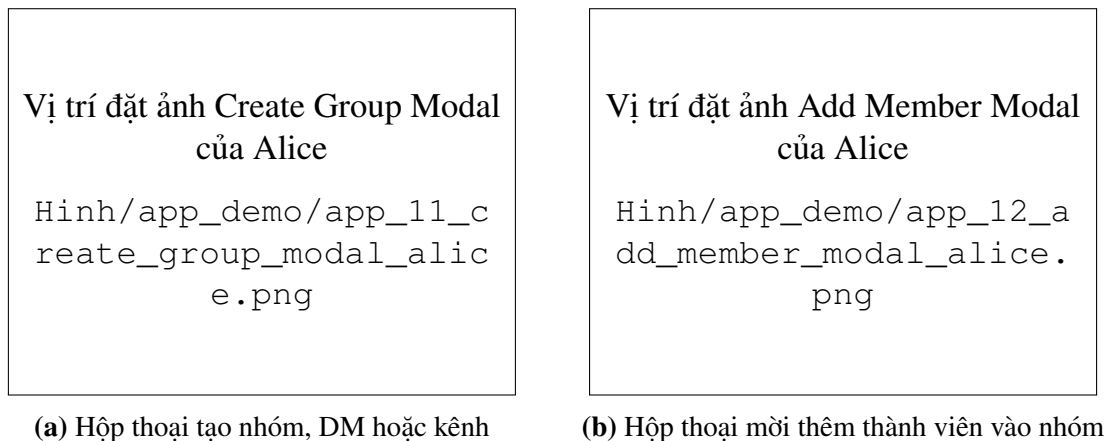


Hình A.5: Bảng hoạt động và không gian kênh của ứng dụng thử nghiệm

Hai màn hình này buộc phần giao diện phải xử lý nhiều kiểu dữ liệu hơn một dòng hội thoại thuần túy, đồng thời cho thấy người làm đồ án đã cố gắng đi xa hơn mức tối thiểu cần có để chứng minh giao thức. Đây là điểm nên được giữ lại trong phụ lục, vì nó phản ánh khá rõ công sức hiện thực của phần ứng dụng.

Việc hình thành nhóm mới và mở rộng thành viên được phản ánh qua các hộp thoại thao tác ở mức người dùng. Hình A.6 minh họa hai bước này, qua đó giúp người đọc quan sát quan hệ giữa việc tạo nhóm, tạo kênh và việc mời thêm thành viên từ danh sách định danh hiện có.

Nhóm các hộp thoại ở Hình A.6 nhìn qua khá nhỏ, nhưng lại là chỗ nổi trực tiếp giữa thao tác người dùng và thay đổi danh sách thành viên của nhóm. Việc tách

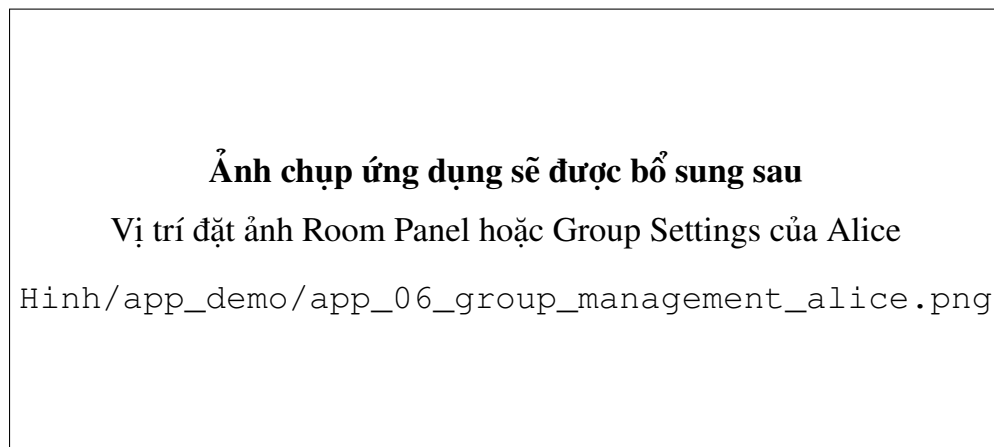


Hình A.6: Các hộp thoại tạo nhóm và thêm thành viên

riêng bước tạo nhóm, tạo kênh và mời thành viên giúp luồng dùng rõ ràng hơn, đồng thời cho thấy phần demo này đã được chăm chút cả ở các thao tác phụ, không chỉ ở màn hình chính.

A.4 Quản trị nhóm và chẩn đoán

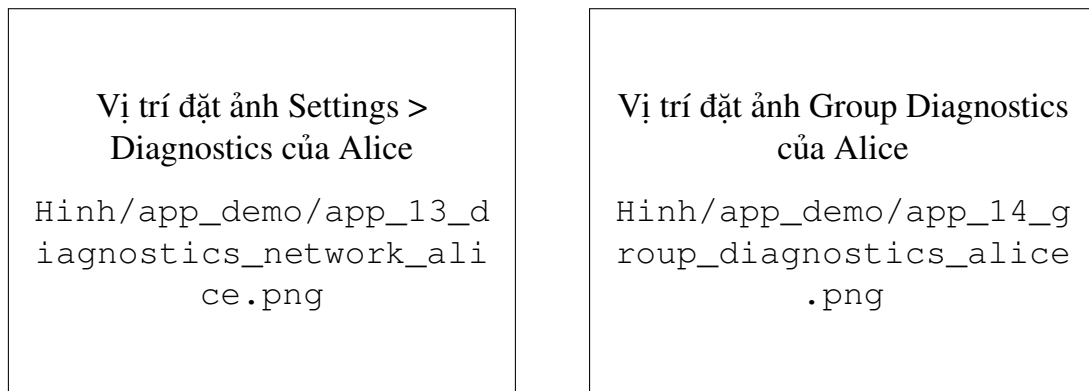
Phần quản trị nhóm là nơi ứng dụng lộ rõ nhất lượng việc ở phía sau. Tại đây, người dùng không chỉ xem danh sách thành viên mà còn phải đọc được vai trò, tình trạng và chính sách mời hiện hành của nhóm. Điều này quan trọng với đề án vì các thay đổi tương như đơn giản ở giao diện thực ra đều chạm vào tiến trình Proposal và Commit ở tầng dưới. Hình A.7 minh họa màn hình quản trị đó.



Hình A.7: Màn hình quản trị thành viên và chính sách mời của nhóm

Cuối cùng, ứng dụng còn dành chỗ cho các thông tin chẩn đoán ở mức mạng và mức nhóm. Hình A.8 minh họa hai màn hình này. Chúng cho phép người đọc quan sát trực tiếp PeerID cục bộ, trạng thái kết nối và một phần thông tin nội bộ của nhóm trong quá trình vận hành.

Các màn hình chẩn đoán không được đưa vào chỉ để làm ứng dụng trông đầy đủ



(a) Màn hình chẩn đoán mạng và trạng thái cục bộ

(b) Màn hình chẩn đoán nhóm hoặc lịch sử fork-heal

Hình A.8: Các màn hình theo dõi trạng thái mạng và trạng thái nhóm

hơn. Trong quá trình làm đồ án, đây là phần khá hữu ích để theo dõi kết nối, trạng thái nhóm và các dấu hiệu bất thường khi thử nhiều nút. Nói cách khác, phần này không trực tiếp làm tăng đóng góp nghiên cứu, nhưng nó cho thấy phần triển khai đã đi tới mức có công cụ tự quan sát và tự kiểm tra, chứ không chỉ dừng ở việc gửi được một vài tin nhắn.

Kết chương. Phụ lục này cho thấy phần ứng dụng desktop của đồ án không chỉ là chỗ đặt thêm ảnh màn hình. Từ luồng vào hệ thống, cấp bundle, tạo nhóm, thêm thành viên, trao đổi tin nhắn cho đến phần hoạt động, quản trị và chẩn đoán, ứng dụng đã được làm thành một chuỗi sử dụng tương đối trọn vẹn. Nhờ đó, phần demo không đứng tách khỏi phần nghiên cứu, mà trở thành nơi để kiểm tra xem các cơ chế điều phối được đề xuất có thật sự đi vào hành vi sử dụng cụ thể hay không.